

EYEQ: A HYBRID QUANTUM–CLASSICAL EYE-GAZE TYPING SYSTEM ON COMMODITY WEBCAMS

FINAL YEAR

PROJECT PHASE-1 REPORT

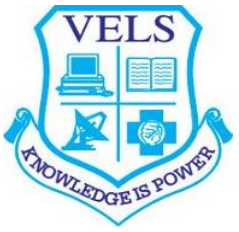
Submitted by

DEVASH. S (23619110)

in partial fulfilment for the award of the degree of

BACHELOR OF TECHNOLOGY CSE –

ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING



DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

SCHOOL OF ENGINEERING

VELS INSTITUTE OF SCIENCE, TECHNOLOGY AND ADVANCED STUDIES

(VISTAS)

CHENNAI

OCTOBER 2026

BONAFIDE CERTIFICATE

Certified that this project **"EYEQ: A HYBRID QUANTUM–CLASSICAL EYE-GAZE TYPING SYSTEM ON COMMODITY WEBCAMS"** is the bonafide work of **DEVASH. S (23619110)** who carried out the project under my supervision. Certified further, that to the best of my knowledge, the work reported here does not form part of any other project report or dissertation on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidate.

SIGNATURE OF THE HOD

Dr. K. Ulaga Priya, M.E., Ph.D.

Associate Professor & Head

Department of AI & ML

School of Engineering

VISTAS

Chennai - 600 117

SIGNATURE OF THE SUPERVISOR

Dr. R. Anandan, M.E., Ph.D.

Professor

Department of AI & ML

School of Engineering

VISTAS

Chennai - 600 117

Submitted for the Project Work and Viva-Voce examination held on …… at Vels Institute of Science, Technology and Advanced Studies (VISTAS), Chennai – 600 117.

INTERNAL EXAMINER **EXTERNAL EXAMINER**

ACKNOWLEDGEMENT

It is certainly a great delight and proud privilege to acknowledge the help and support I received from the positive minds around me in making this venture a successful one. The infrastructural support with all kinds of laboratory facilities has been a motivating factor in the completion of this project work, and for this I express my sincere gratitude to our **FOUNDER & CHAIRMAN, Shri. Dr. ISHARI K. GANESH**. I am grateful to our **HEAD OF THE DEPARTMENT, Dr. K. ULAGA PRIYA**, for continuous support of the project, from initial advice in the early stages of conceptual inception through ongoing encouragement to this day. I extend my warmest thanks to my **INTERNAL GUIDE, Dr. R. ANANDAN**, who has patiently guided the work and provided invaluable advice at every stage, from the problem statement to the evaluation and the conference manuscript.

I am deeply thankful to **V. MARUTHI**, Research Scholar, Department of Computer Science and Engineering, National Institute of Technology Tiruchirappalli, for mentorship throughout the research, and to the **National Institute of Technology Tiruchirappalli** for providing access to its laboratory facilities, under which this work was initiated. I thank the nine volunteers who gave their time to the gaze recording sessions; every accuracy figure in this report rests on their patience.

I would like to express my gratitude to the staff members, the librarian and the non-teaching staff, who have helped and supported me throughout. Last but not least, I wish to thank my parents and my friends for their support and interest, which encouraged me to pursue this work in my own way.

DEVASH S 23619110

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	ABSTRACT	i
	LIST OF FIGURES	ii
	LIST OF ABBREVIATIONS	iii
1	**INTRODUCTION**	1
	1.1 Overview of the Project	1
	1.2 Scope and Objective	8
2	**LITERATURE REVIEW**	15
	2.1 Literature Overview	15
	2.2 Literature Survey	15
3	**SYSTEM DESIGN**	24
	3.1 Existing System	24
	3.2 Proposed System	25
	3.3 System Architecture	27
	3.4 Hardware Requirements	32
	3.5 Software Requirements	33
4	**IMPLEMENTATION**	35
	4.1 Methodology	35
	4.2 Implementation	39
	4.2.1 Landmark Acquisition and Canthus Feature Module	40
	4.2.2 Calibration and Gaze Mapping Module	43
	4.2.3 Filtering, Dwell and Key Decision Module	46
	4.2.4 Quantum Decision Module	49
	4.2.5 Quantum Residual Gaze Correction Module	54
5	**CONCLUSION AND FUTURE WORK**	57
	5.1 Conclusion	57
	5.2 Future Work	62

CHAPTER NO.	TITLE	PAGE NO.
	5.3 Source Code	64
	5.4 Results	73
	REFERENCES	97

ABSTRACT

Text entry still assumes working hands and a physical keyboard, which excludes people with severe motor impairment. This project presents EYEQ, a hybrid quantum–classical eye-gaze typing system that runs on a commodity webcam and a web browser, with no infrared hardware. MediaPipe face and iris landmarks are mapped to a continuous screen coordinate through a nine-target ridge-regression calibration; an adaptive One Euro filter, dwell hysteresis and a calibrated confidence gate turn that coordinate into committed characters, and a language model offers word completions. Measuring gaze as a continuous angle on held-out targets exposed a defect in a step the literature treats as settled: the vertical iris feature is normalised by the eyelid aperture, which moves with every squint and blink. EYEQ instead expresses the iris offset in a frame fixed to the inner and outer corners (canthi) of each eye. On eleven sessions from nine participants this reduced held-out gaze error from 5.55° to 3.76° (−32.3 %, $t(8) = 9.31$, $p = 0.000014$), improved every participant and more than halved the vertical error. For the key decision the project formulates a quantum conditional selection rule that provably contains conventional log-linear fusion; measured at matched prompt rate it changes accuracy by a median −0.03 percentage points, because coherence is anti-correlated with correctness. A quantum-kernel movement gate with a quantum Bayesian reliability model cut wrong keystrokes 38-fold in simulation, a QAOA word re-decoder offers corrections, and a six-qubit residual-correction register reduced gaze error by a further 4.2 % for every participant, though not yet significantly against a size-matched classical model. All quantum components run as exact classical simulation and are adopted only where they beat a classical counterpart. Two short live typing trials reached only 18–33 % commit accuracy, and a separate keyboard recording identified posture drift after calibration as the dominant live error, the first task of the second phase.

Keywords

Eye-gaze typing, webcam gaze estimation, canthus-relative normalisation, assistive text entry, hybrid quantum –classical machine learning, quantum conditional selection rule, QAOA, quantum Bayesian network, dwell selection, calibration.

LIST OF FIGURES

FIGURE NO.	TITLE	PAGE NO.
Figure 1	The EYEQ keyboard running live on a commodity webcam	2
Figure 2	The two problems in series	4
Figure 3	Cost of the gaze-sensing hardware on a logarithmic scale	11
Figure 4	Directional accuracy reported by Waideman and Aquino Junior, split by axis	20
Figure 5	System block diagram of the EYEQ webcam eye-gaze typing system (100)	28
Figure 6	Flowchart of the gaze-typing process, steps S1–S12, including calibration (S5) and the quantum residual correction stage (S7–S9)	29
Figure 7	Project timeline, June to October 2026	36
Figure 8	The 21 target positions of an evaluation session on the recording display: 13 fitting targets and 8 held-out targets spanning corners, edges and centre	37
Figure 9	Landmarks used by EYEQ	40
Figure 10	Eyelid-relative against canthus-relative normalisation of the iris offset	41
Figure 11	The nine calibration targets, anchored to key centres: Q, P, A, G, L, Z, V, M and the midpoint of T and Y, since the top row has no middle key	44
Figure 12	Stages of the quantum conditional selection rule	49
Figure 13	The six-qubit correction-hypothesis register	55
Figure 14	Output 1 — live word suggestions	73
Figure 15	Output 2 — QAOA word re-decoding	74
Figure 16	Output 3 — ask, never guess	74
Figure 17	Held-out error by feature set over the nine participants	75
Figure 18	Held-out error per participant under the deployed and the proposed features	76
Figure 19	Held-out predictions for one typical session (participant P02) under the two feature sets	77
Figure 20	Held-out error split by axis, participant level (each grey line is one participant; whiskers are one standard deviation)	77
Figure 21	Decomposition of gaze error by head roll on the pilot recordings (63 target groups)	79
Figure 22	Gaze error against the number of calibration targets	82
Figure 23	Top: the selection rule traces the same accuracy/prompt-rate frontier as the log-linear fusion it generalises, at two noise levels	85
Figure 24	Commit accuracy against entry rate for this system (blue, labelled “quantum-inspired”) and an independent classical implementation (gold), on identical sentences, geometry and noise; points are the five noise levels	86
Figure 25	Held-out error per participant for the uncorrected ridge, the classical twin and the quantum register (both ungated)	89

LIST OF ABBREVIATIONS

Abbreviation	Full Form
AUC	Area Under the ROC Curve
CNN	Convolutional Neural Network
CNOT	Controlled-NOT gate
CPM	Characters Per Minute
CSS	Cascading Style Sheets (CSS pixel)
DFD	Data Flow Diagram
EAR	Eye Aspect Ratio
ECE	Expected Calibration Error
FPGA	Field-Programmable Gate Array
GPU	Graphics Processing Unit
HOG	Histogram of Oriented Gradients
I-VT	Velocity-Threshold Identification
LOPO	Leave-One-Participant-Out
LOTO	Leave-One-Target-Out
MAP	Maximum a Posteriori
MLP	Multilayer Perceptron
OLS	Ordinary Least Squares
pp	Percentage points
QAOA	Quantum Approximate Optimisation Algorithm
QBN	Quantum Bayesian Network
QNN	Quantum Neural Network
QRGC	Quantum Residual Gaze Correction
QSVM	Quantum Support Vector Machine
QUBO	Quadratic Unconstrained Binary Optimisation
RBF	Radial Basis Function
SVM	Support Vector Machine
VISTAS	Vels Institute of Science, Technology and Advanced Studies
VQC	Variational Quantum Classifier
WASM	WebAssembly
WPM	Words Per Minute

CHAPTER 1. INTRODUCTION

1.1 OVERVIEW OF THE PROJECT

Introduction

EYEQ is an on-screen keyboard that a person operates by looking at it. It needs no special hardware: an ordinary laptop webcam, a web browser and a short calibration are sufficient. The user rests their gaze on a key for about seven-tenths of a second and the key is typed; a language model offers word completions above the keyboard, so a word rarely has to be spelled out letter by letter. Figure 1 shows the system running live.

Every frame captured by the webcam is passed to a face-landmark model that returns 478 points on the face, including a five-point ring around each iris. From these points the system builds a twelve-component feature vector, maps it to a screen coordinate through a per-user ridge regression, smooths the coordinate with an adaptive One Euro filter, and decides — after a dwell interval — which character the user intended. The decision fuses the gaze evidence with a language prior and is guarded by a confidence threshold: when the evidence is not strong enough, the keyboard asks the user to choose between the leading candidates instead of guessing.

The project treats gaze typing as two problems in series, and they turn out to be different kinds of problem. The first is *estimation*: turning a low-resolution image of a face into a point on the screen. The second is *decision*: turning a noisy point and the sentence typed so far into a committed character. Both were studied, and the balance between them was the most instructive finding of the work.

On the estimation side, measuring gaze as a continuous angle on held-out targets exposed a defect in a step the field treats as settled: the conventional vertical iris feature is normalised by the eyelid aperture, which moves whenever the user squints or blinks. EYEQ instead measures the iris against the **canthi**, the inner and outer corners of the eye, which do not move. On eleven sessions from nine participants this reduced held-out gaze error from 5.55° to 3.76° (−32.3 %) and improved every participant — the project's principal result.

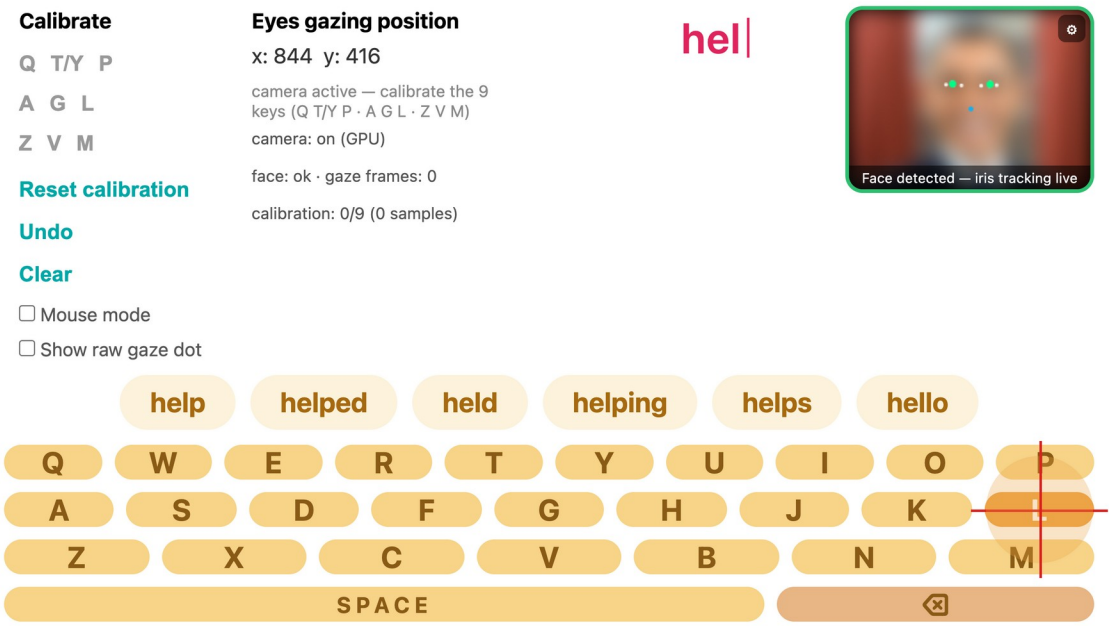


Figure 1 – The EYEQ keyboard running live on a commodity webcam

The preview panel (top right) shows the iris centres (green) and the medial and lateral canthi (white) being tracked; the crosshair rests on the key L, which is filling its dwell timer; the user has gaze-typed "hel" and the language model offers six completions. The face is blurred for privacy; the tracking overlay is drawn on a separate layer and is unaffected.

On the decision side, the project introduces a **quantum conditional selection rule** that places candidate keys in superposition and conditions them on the language context through both magnitude and phase. It provably contains the conventional fusion as a special case and, measured honestly against it, adds nothing, for a

precise and diagnosable reason. Around it a hybrid decision layer and a six-qubit residual-correction register were built and evaluated.

Problem Statement

Text entry still assumes working hands and a physical keyboard, which excludes people with severe motor impairment — for whom the eyes are frequently the last reliably controllable channel [1]. Commercial infrared trackers solve the sensing problem at a price out of reach of most of the people who need them, while most webcam keyboards select keys by blinking or scanning coarse regions and report region-classification percentages rather than whether a user could point at a key.

The problem addressed by this project is therefore stated as follows. *Given a stream of frames from an uncalibrated commodity webcam, the system must (i) estimate the continuous on-screen point at which the user is looking, and (ii) commit the character the user intends, with the fewest possible wrong keystrokes, in real time and entirely on the user's own device, without infrared illumination or any additional hardware.*

Figure 2 separates the two sub-problems. Each carries its own difficulties:

- **Low-resolution sensing.** At a typical viewing distance the iris spans only a handful of pixels, so landmark positions jitter from frame to frame.
- **Contaminated features.** The conventional vertical feature divides the iris height by the eyelid aperture, so squinting, partial blinks and lighting changes alter the feature although the eye has not moved.

- **Head motion.** Features measured along image axes change when the head turns or tilts; most existing systems ignore head pose or forbid head movement altogether.
- **Sparse calibration.** A user will tolerate only a short calibration — nine targets in this system — so the gaze map must be fitted from very little data.
- **Gaze error comparable to key size.** The gaze error is 1.0–1.7 times the width of a single QWERTY key, so a naive decision rule commits many wrong characters.
- **Real-time constraint.** Everything must fit inside a 33 ms frame budget, and no stage may delay the dwell interval the user experiences.

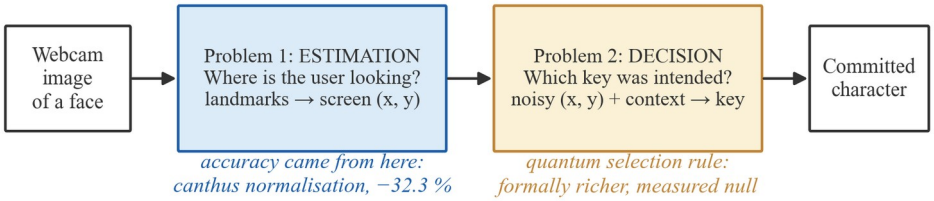


Figure 2 – The two problems in series

Estimation turns an image into a screen coordinate; decision turns a noisy coordinate and the language context into a committed key. In this project the accuracy came from the first; the quantum selection rule addressed the second and returned a measured null.

Objectives of the Project

The project set out to achieve the following objectives.

1. **Continuous gaze estimation on ordinary hardware** — estimate a continuous screen coordinate from webcam landmarks through a short per-user calibration, with no infrared hardware.

2. **Remove the dominant systematic error** — diagnose why vertical gaze is less accurate than horizontal gaze in webcam systems, and design a feature that is immune to the cause.
3. **Reliable, fatigue-aware selection** — commit characters by dwell with hysteresis, and ask the user instead of guessing whenever the evidence is weak.
4. **Investigate quantum decision algorithms rigorously** — design a quantum conditional selection rule, prove its relation to the conventional rule, and build a hybrid quantum–classical decision layer.
5. **Honest, reproducible evaluation** — record real participants and report accuracy on held-out targets, with the participant as the unit of analysis.
6. **Real-time, private operation** — keep the pipeline inside the webcam frame budget on an ordinary CPU, with all gaze computation on the user's device.
7. **Record negative results** — document every idea that was implemented, measured and rejected.

Scope of the Project

The project covers the design, implementation and evaluation of a complete software system: a browser client that performs landmark tracking, calibration, filtering and dwell selection, and a local Python server that provides word suggestions, the language prior and the research modules. The keyboard is a 26-letter QWERTY layout with space and backspace, operating over a 16,839-word English vocabulary. Evaluation covers eleven recorded calibration sessions from nine participants and end-to-end typing simulation at five controlled noise levels; two short live typing trials were also recorded for the third review. The detailed functional, technical and application scope is set out in Section 1.2.

System Overview

The system is organised in three layers: an **estimation layer** that converts landmarks into a twelve-component feature vector and a coarse gaze estimate by ridge regression; an **interaction layer** that filters the estimate and decides by dwell with hysteresis when a key has been selected; and a **decision layer** that fuses gaze and language evidence, decides whether to commit or ask, and re-decodes finished words. The browser client carries everything on the real-time path, so the keyboard keeps working if the server is unavailable; the server provides the language model, suggestions, corrections and the quantum research modules. Section 3.2 describes the layers in detail.

Advantages of the System

- **No special hardware.** The system runs on a webcam costing about Rs. 1,900, against about Rs. 48,000 for a consumer infrared tracker.
- **Continuous pointing at individual keys.** Keys are selected by looking at them, not by blinking or scanning through groups.
- **An eyelid-independent vertical feature.** Canthus normalisation more than halved the vertical gaze error (−53.2 %) for every participant.
- **Head-roll invariance by construction.** No roll angle is estimated and none is needed.
- **Ask, never guess.** Below a calibrated confidence threshold the keyboard offers the leading candidates as choice chips instead of committing a possibly wrong key.
- **Calibrated confidence.** Platt scaling reduced the expected calibration error of the commit confidence from 0.0650 to 0.0167.

- **Privacy.** All gaze computation runs on the user's device; no video is stored.
- **Graceful degradation.** Every server-side component is optional, and a watchdog disables any gate that refuses five keystrokes in a row.

Applications of the Project

- **Augmentative and alternative communication.** The intended primary application is text entry for people with severe motor impairment. The system has not yet been evaluated with motor-impaired users; that evaluation is a Phase 2 requirement.
- **Hands-busy and sterile settings.** Operating theatres, laboratories and workshops, where the hands are occupied or must stay sterile.
- **Low-cost assistive deployment.** Schools, hospitals and homes where an infrared tracker is unaffordable but a laptop with a webcam is already available.
- **Research platform.** The de-identified recordings, benchmarks and test suites allow gaze-estimation and text-entry methods to be compared on held-out targets in degrees of visual angle.
- **A testbed for quantum machine-learning claims.** Every quantum component is paired with a classical counterpart, so the system can be used to test whether a proposed quantum method adds anything measurable.

Expected Outcomes

The first phase was planned to deliver a working webcam gaze keyboard, a measured gaze accuracy in degrees on unseen targets, a diagnosis of the dominant error source, a rigorously evaluated quantum decision layer, a conference manuscript and a patent invention disclosure. Section 5.1 reports the status of each

against the objectives; all were delivered, and the live typing trials identified the main obstacle remaining for the second phase.

Future Enhancements

The principal enhancements planned for Phase 2 are an eyelid-perturbation experiment to establish the canthus mechanism causally, axis-specific feature sets, confirmation of the residual register on fresh recordings, posture-robust calibration with automatic re-centring, broader recruitment including spectacle wearers, and a larger live typing study. They are described in Section 5.2.

1.2 SCOPE AND OBJECTIVE

Purpose of the Study

The purpose of the study is to establish whether a keyboard driven only by a commodity webcam can support continuous pointing at individual keys, to find what limits its accuracy, and to determine — by measurement rather than assertion — whether quantum decision algorithms improve the choice of the intended key when compared fairly against the classical rule they would replace.

General Objective

To design, implement and evaluate a webcam-only, real-time eye-gaze typing system whose gaze accuracy is reported in degrees of visual angle on held-out targets, and whose every quantum component is adopted only if it outperforms its classical counterpart on held-out data.

Specific Objectives

1. Report held-out gaze error in pixels, degrees of visual angle and fractions of a key pitch, on targets never used for fitting.
2. Reduce the vertical gaze error by replacing the eyelid-normalised vertical feature with a feature measured against a rigid anatomical baseline.
3. Keep the per-frame pipeline inside the 33 ms frame budget, with no quantum stage on the frame or dwell path.
4. Commit a key only when the fused, calibrated confidence reaches $\theta = 0.80$, and offer choice chips otherwise.
5. Prove that the quantum selection rule contains the conventional log-linear fusion, and compare the two at matched prompt rate.
6. Adopt any new component only if it beats the incumbent on held-out data, paired over participants, at $p < 0.05$.

Scope of the Project

A. Functional Scope

- Nine-target calibration anchored to key centres, stored in the browser so that it is performed once per device.
- Continuous gaze tracking with a visible crosshair, a dwell ring and live word suggestions.
- Dwell typing of letters, space and backspace, with choice chips for uncertain keys and correction chips for finished words.
- A capture tool that records de-identified evaluation sessions, and a live evaluation mode that logs every committed keystroke.

B. Technical Scope

- A JavaScript browser client using MediaPipe Tasks Vision for landmarks, with ridge regression solved in the browser.
- A Python FastAPI server for the language model, suggestions, corrections, the movement gate and QAOA word re-decoding.
- Quantum components implemented in PennyLane and PyTorch and executed by exact classical state-vector simulation.
- Evaluation scripts, figure generators and test suites for every reported number.

C. Application Scope

The system is evaluated for English text entry on one display at one nominal viewing distance. Several things are deliberately out of scope for this phase: no infrared or special-purpose hardware is used; no quantum hardware is used and no quantum speed-up is claimed; spectacle wearers were not recorded; and users with unstable fixation (nystagmus, strabismus or saccadic intrusions) are outside the population the method is designed to serve.

Importance of the Project

Research-grade infrared eye trackers reach sub-degree gaze accuracy with dedicated hardware [2], but at a price that excludes most of the people who need them. The approximate prices compiled for the project's zeroth review were about Rs. 9.6 lakh for a wearable research-grade tracker (Tobii Pro Glasses) and about Rs. 48,000 for a consumer screen-mounted tracker (Tobii Eye Tracker 5), against about Rs. 1,900 for a commodity webcam; prices vary by market and over time, but the gap of two to three orders of magnitude does not. EYEQ runs on hardware

that most users already own, as Figure 3 illustrates. Moving from coarse regions to continuous pointing at individual keys is what makes a full keyboard usable without slow scanning, and it is what makes the accuracy of the gaze estimate, measured in degrees, the quantity that matters.

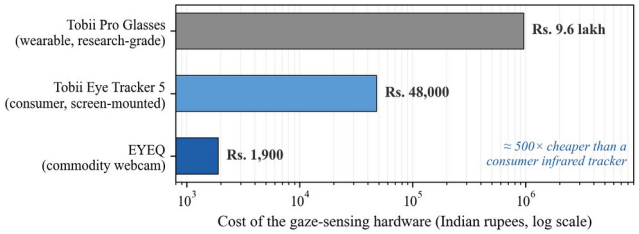


Figure 3 – Cost of the gaze-sensing hardware on a logarithmic scale

A commodity webcam is roughly 500 times cheaper than a wearable research-grade tracker and 25 times cheaper than a consumer infrared tracker, at the approximate prices compiled for the project's zeroth review.

System Functionality Overview

1. **Calibration.** The user fixates nine targets anchored to key centres and clicks each five times; the ridge map is fitted in the browser.
2. **Tracking.** Every frame yields landmarks, the twelve-component feature vector and a filtered screen coordinate shown as a crosshair.
3. **Selection.** Resting the gaze on a key fills a dwell ring; after 693 ms the key is selected, judged on the centroid of the dwell window.
4. **Decision.** The gaze posterior over keys is fused with the language prior; the key is committed above the confidence threshold, otherwise choice chips are offered.
5. **Suggestions and corrections.** Word completions are refreshed after every letter; at each space the finished word is re-decoded and a correction chip may be offered.

Methodology for Achieving Objectives

The work followed a cycle of *build, instrument, record, measure, adopt*. Each component was first implemented in the simplest form that could be measured; the system was instrumented so that every decision could be logged and replayed; real gaze was recorded from participants; and every candidate improvement was measured against the incumbent on the same recordings before it was allowed to change the typing path. Section 4.1 describes the methodology and the evaluation protocol in detail.

Hardware and Software Requirements

The system needs only a computer with a modern browser and a 720p or 1080p webcam. The client uses MediaPipe Tasks Vision 0.10.14 in the browser; the server uses Python 3.13 with FastAPI, NumPy, SciPy, scikit-learn, PennyLane and PyTorch. The complete requirements are listed in Sections 3.4 and 3.5.

Benefits of the Proposed System

For a user, the system offers a writing channel at the cost of a webcam. For researchers, it offers de-identified recordings, benchmarks that report error in degrees on held-out targets, and a documented record of negative results.

Challenges and Limitations

Several challenges shaped the work. Landmark jitter at webcam resolution makes single-frame estimates unreliable, which is why the dwell centroid and the One Euro filter are load-bearing. An early movement gate received raw rather than normalised eye openness and refused every keystroke; the fix and a watchdog are covered by regression tests. In the two live typing trials, posture changes after

calibration moved every estimate by a common offset. The limitations of the results are stated in Section 5.1.

Novelty and Contributions

The novelty of EYEQ lies in one new representation, two new quantum formulations, and an evaluation discipline that let each claim be tested rather than asserted. Table 1 summarises the contributions and the evidence behind each.

Table 1 – Summary of the contributions of the project and the evidence supporting each

#	Contribution	Nature	Key evidence
1	Canthus-relative normalisation of the iris offset, in the eye's own frame	New feature representation	Held-out error 5.55° → 3.76° (−32.3 %); all 9 participants improved; $t(8) = 9.31$, $p = 1.4 \times 10^{-5}$
2	Head-roll invariance by construction	Property of the representation	Feature drift $\leq 2 \times 10^{-15}$ over 0–40° of roll, against 0.069 for an image-axis offset
3	Diagnosis of a reported but unexplained vertical asymmetry	Analytical insight, confirmed by measurement	Vertical error 4.31° → 2.02° (−53.2 %, 9/9); horizontal not improved (+5.2 %)
4	Quantum conditional selection rule with containment and expressiveness propositions	New decision formulation	Reduces to log-linear fusion to 10^{-12} at zero phase
5	A measured null with a diagnosed mechanism	Scientific finding	Median −0.03 pp at matched prompt rate; coherence anti-correlated with correctness
6	Quantum residual gaze-correction register with Born-rule expected-value readout	New estimation stage (patent disclosure)	3.765° → 3.607° (−4.2 %), 9/9 participants, $p = 0.0004$
7	Calibration-size inversion in model selection	Methodological finding	Ridge best at the deployed 9 targets; kernel ridge best only beyond 12
8	A complete record of negative results, with mechanisms	Methodological contribution	Quantum kernels, head pose, iris radius, amplification, hierarchical selection

Two features distinguish this work from the surveyed literature: it reports gaze error as a continuous angle on held-out targets, which is what made the eyelid

defect visible, and it places every quantum component inside a family that contains the classical incumbent, so that each comparison measures the quantum mechanism and nothing else.

Organisation of the Report

Chapter 2 surveys webcam gaze keyboards, iris-localisation methods, selection strategies and the relevant quantum machine-learning literature, and identifies the research gap. **Chapter 3** describes the existing and proposed systems, the architecture, data flow, modules and requirements. **Chapter 4** presents the methodology, the evaluation protocol and the implementation of every module. **Chapter 5** states the conclusions and limitations, outlines the future work, presents source-code excerpts and reports the results. The references follow.

CHAPTER 2. LITERATURE REVIEW

2.1 LITERATURE OVERVIEW

Eye typing has a long history in assistive technology. Early systems relied on dedicated infrared trackers and established the core interaction vocabulary still in use today: dwell-time selection, on-screen keyboards with enlarged targets, and word prediction to reduce the number of selections per word [1]. The central interaction difficulty was identified early as the *Midas touch* problem — the eyes are used both to look and to act, so anything the user merely inspects is at risk of being selected [3]. Dwell time, blink selection and scanning are the three classical answers to it.

Gaze estimation has been studied under two broad approaches [2]. *Model-based* methods fit a geometric eye model, usually from infrared corneal reflections, and reach sub-degree accuracy with dedicated hardware. *Appearance-based* methods regress gaze from image pixels, made practical for consumer cameras by datasets such as MPIIGaze [4], GazeCapture [5] and EYEDIAP [6]. Most webcam keyboards — and EYEQ — sit between the two: they derive a few geometric features from facial *landmarks* and map them to the screen with a short per-user calibration.

2.2 LITERATURE SURVEY

The survey is organised by theme: how existing webcam keyboards localise the iris and select keys, how gaze and language evidence are combined, which quantum algorithms are relevant to the decision problem, and one reported observation that motivates the main method of this project.

Families of Iris Localisation

Threshold, projection and circle fitting. Saraswati et al. [7] locate the iris by binary thresholding and integral projection; it is inexpensive, but the threshold must be retuned for each lighting condition and detection fails beyond 50 cm. Padilla et al. [8] fit the iris as a circle with Sobel edges and a circular Hough transform on an FPGA — the only surveyed work to target dedicated hardware.

Landmark regression with aspect ratios. The largest group [9], [10], [11], [12] uses the 68-point *dlib* shape predictor and derives an eye aspect ratio (EAR) [13] or a horizontal/vertical gaze ratio. One property of this family deserves emphasis: the 68-point model contains *no iris landmarks at all*. It locates the eye corners and lids, and everything concerning the pupil is inferred from those. This is why all four systems select keys by blinking or scanning rather than by pointing — their representation cannot support a continuous coordinate.

Dedicated iris landmarks. Rafid et al. [14] and the virtual-mouse system of A. SK et al. [15] use MediaPipe Face Mesh [16], which supplies a five-point iris ring per eye in addition to the 468 face landmarks. This yields the iris centre directly and is the prerequisite for continuous regression. Only two of the ten surveyed works adopt it, both dated 2023 or later. EYEQ builds on the same landmark model.

End-to-end learning. Waideman and Aquino Junior [17] regress gaze from pixels with a four-stream convolutional network and are the only surveyed work to model head pose, supplying head position as a fourth network channel; training required two V100 GPUs. Abhaya et al. [10] classify nine gaze directions with a CNN.

Eye-corner reference vectors. Outside the keyboard literature, Sesma, Villanueva and Cabeza [18] evaluated the pupil-centre-to-eye-corner vector as a webcam gaze feature. It is the closest prior idea to the representation introduced in this project. EYEQ differs in three respects: it uses *both* canthi to define a midpoint and an axis, it resolves the iris offset along that axis and its perpendicular so that the feature rotates with the head, and it scales by the inter-canthal distance so the feature is dimensionless. The second property is what gives head-roll invariance by construction (Section 4.2.1).

Browser-Based Gaze Tracking

Gaze tracking inside a web browser was made practical by WebGazer [19], which runs entirely in a web page, locates the eyes with an interchangeable detector and maps eye features to screen coordinates by ridge regression. Its distinguishing idea is passive self-calibration: clicks and cursor movements on the page are treated as moments when the user is probably looking at the pointer and are added as training samples, so the gaze map keeps improving during normal use. EYEQ inherits two choices from this line of work and qualifies both. It too fits a ridge map in the browser, but over landmark-derived geometric features rather than eye-patch appearance, which keeps the model to 24 numbers. It too admits ordinary clicks as refinement samples, but at one-tenth of the weight of a calibration click and with coverage-aware eviction (Section 4.2.2), because interaction samples are unverified and cluster wherever the user happens to be typing. WebGazer was designed for gaze-aware web pages rather than for typing on a keyboard, where the target is a single key and a wrong selection must be undone.

Selection Strategies and Language Models

Once a gaze estimate exists, a key must be selected. **Blink selection** avoids the Midas touch but turns a natural reflex into a command and causes fatigue; **scanning** removes the need for accurate pointing but is inherently slow — Islam et al. [9] report 9.5 characters per minute. **Dwell selection** commits a target after the gaze has rested on it for a fixed interval. Li et al. [20] address the Midas touch with an adaptive dwell time that shortens as the user's input speed rises, and with a three-way decision that splits each key into positive, boundary and negative regions; boundary cases, however, are pushed back to the user rather than resolved.

Fixations are classically separated from saccades by velocity-threshold identification (I-VT) [21], and blinks by the eye aspect ratio [13]; EYEQ's movement gate replaces such hand-tuned thresholds with a learned classifier (Section 4.2.4).

Language models are the second lever. Enlarging targets and recovering the intended word with a lexicon is a standard trade in text entry, as in SHARK² [22]. Waideman et al. [17] recommend the next word with a trigram model, but it cannot correct a mis-selected key. Text-entry rates and errors follow established conventions [23].

The usual way of combining gaze and language is a product of a gaze posterior and a language prior. As Chapter 4 shows, that product is a product of *marginals*, and it discards how the linguistic evidence for each letter is distributed across candidate words. Whether retaining that structure helps is one of the questions this project answers.

Quantum and Quantum-Inspired Machine Learning

Several quantum algorithms are relevant to the decision side of gaze typing, and the literature is equally clear about where they do and do not offer an advantage.

Amplitude amplification. Grover's search [24] and amplitude amplification [25] rotate probability mass into a marked subspace in $O(1/\sqrt{p})$ rather than $O(1/p)$ steps — useful only when the marked mass p is small.

Quantum kernels. The fidelity $|\langle \phi(x)|\phi(z)\rangle|^2$ between data embedded by a quantum feature map defines a kernel for a support vector machine; the ZZ feature map of Havlíček et al. [26] is conjectured hard to evaluate classically. Fidelity kernels, however, *concentrate exponentially* with qubit count [27]: the Gram matrix approaches the identity and the model can only memorise. Angle-encoding kernels are classically simulable.

Variational circuits. Parameterised circuits trained by gradient descent act as classifiers or regressors; data re-uploading provably increases the expressivity of a fixed number of qubits [28], while *barren plateaus* — gradients vanishing with width and depth [29] — are mitigated by shallow circuits and small-angle initialisation. PennyLane [30] differentiates such hybrid models.

Quantum approximate optimisation. QAOA [31] alternates problem and mixing Hamiltonians to approximate the ground state of a QUBO; an XY mixer [32] keeps one-hot constrained problems feasible. No general advantage is known at the sizes considered here.

Confidence calibration. A threshold is meaningful only if the confidence it tests is calibrated. Platt scaling [33] and temperature scaling [34] repair over-confidence

without changing which candidate wins; expected calibration error (ECE) measures the remaining gap.

The project adopts the discipline this literature implies: a quantum component is worth adopting only if it beats the strongest classical alternative on held-out data.

The Vertical-Asymmetry Observation

The observation that motivates the main method of this project is present in the surveyed literature but not commented upon there. Waideman and Aquino Junior [17] report directional accuracy split by axis: 93 % for left/right and 76 % for up/down (Figure 4). The gap is large and the paper does not explain it.

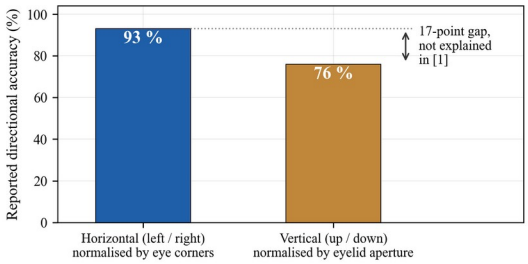


Figure 4 – Directional accuracy reported by Waideman and Aquino Junior, split by axis

The vertical channel — the one conventionally normalised by the eyelid aperture — is 17 points worse.

Consider how the vertical channel is conventionally formed. Let p be the iris centre and let t and b be the upper and lower eyelid landmarks. The standard vertical feature is

$$v = \frac{p_y - t_y}{b_y - t_y + \epsilon}, \tag{1}$$

that is, the iris height expressed as a fraction of the *eyelid aperture*. The denominator of Equation (1) is not a fixed anatomical quantity: it contracts with every squint, every partial blink and with ambient brightness. Any such change alters v without the eye having moved. The horizontal channel, normalised by the distance between the eye corners, suffers no equivalent contamination. An eyelid-normalised vertical channel should therefore be measurably worse than its horizontal counterpart — which is exactly what is reported. The method of this project follows directly: normalise the iris offset against a structure that does not move.

Comparative Analysis of Webcam Gaze Keyboards

Table 2 compares ten recent works on webcam gaze keyboards and closely related interfaces, grouped by how the iris is localised. The rightmost column is the point of the table: the reported figures measure region classification or text-entry rate, not continuous gaze error, and are therefore not directly comparable with this work.

Table 2 – Survey of ten recent webcam gaze-keyboard systems

Work	Face detector	Iris localisation	Selection	Head pose	Reported
Waideman et al. [17]	MMOD-CNN	4-stream CNN (face, both eyes, head)	CNN regression	Yes, 4th channel	93 % L/R, 76 % U/D
Islam et al. [9]	HOG + SVM	dlib 68-point, midpoint	Blink, sequential scan	Movement prohibited	CPM / WPM only
Abhaya et al. [10]	dlib	dlib 68-point landmarks	CNN, 9 directions (T9)	No	15.3 % error
Rafid et al. [14]	MediaPipe	MediaPipe iris	2 × 4 grid, blink	No	14.53 CPM
Varshini et al. [11]	dlib	dlib 68-point, EAR	Blink, group scan	No	92.5 %, 17 WPM
A. SK et al. [15]	MediaPipe	MediaPipe Face Mesh iris	Blink duration	Cited, not implemented	—

Work	Face detector	Iris localisation	Selection	Head pose	Reported
Kathpal et al. [12]	OpenCV / dlib	Horizontal / vertical gaze ratio	Blink	No	91 % / 85 % (glasses)
Saraswati et al. [7]	Haar cascade	Threshold + integral projection	Pointer, blink	No	40–50 cm only
Padilla et al. [8]	—	Sobel + thinning + circular Hough	On-screen keyboard	No	FPGA ≡ MATLAB
Li et al. [20]	—	SMI ETG 2w (commercial infrared)	Adaptive dwell, 600 ms	Noted, unsolved	—
EYEQ (this work)	**MediaPipe**	**MediaPipe iris, canthus-normalised**	**Dwell pointing**	**Implicit (canthus frame)**	**3.76° continuous**

Reported figures measure region classification or entry rate rather than continuous gaze error.

Two observations follow. First, **no surveyed work reports gaze error in degrees of visual angle**: the percentages measure region classification, often over two or three zones where chance alone yields 33–50 %, so this project's results cannot be ranked against them and no such ranking is attempted. Second, **head pose is almost entirely absent.** One work models it implicitly as a network channel [17]; one prohibits head movement and reports the resulting discomfort as a limitation [9]; one observes that head shaking causes error and leaves it unaddressed [20]. No surveyed work performs geometric head-pose estimation.

Research Gaps Identified

The research gap is twofold: no webcam keyboard has examined *what the iris offset is normalised against*, and the conventional product-of-marginals fusion has never been compared against a strictly richer rule under conditions that would reveal whether the discarded structure matters. Table 3 collects the limitations identified in this review and how EYEQ addresses each.

Table 3 – Limitations of existing work and how the proposed system addresses them

Limitation in existing work	Consequence	How EYEQ addresses it
68-point models contain no iris landmarks	Keys selected by blink or scan, not by pointing	MediaPipe iris ring supports a continuous coordinate
Vertical feature normalised by eyelid aperture	Vertical channel contaminated by squint, blink and lighting	Canthus-relative normalisation against a rigid baseline
Head pose ignored or movement forbidden	Error with head movement; discomfort	Canthus frame: roll-invariant by construction
Region-classification metrics	Results not comparable; pointing ability untested	Continuous error in degrees on held-out targets
Language model predicts, cannot correct	Mis-selected keys persist	Gated fusion, chips, whole-word re-decoding
Fixed, hand-tuned thresholds	Not user-adaptive; brittle under drift	Learned movement gate with per-user baseline and watchdog
Quantum methods without classical controls	Unverifiable accuracy claims	Containment; matched prompt rate; pre-registered adoption

Summary of Literature Findings

The literature establishes three things. First, dedicated iris landmarks — available from MediaPipe and adopted by only two of the ten surveyed keyboards — are the prerequisite for continuous pointing; systems built on the 68-point model can only blink or scan. Second, evaluation practice is the largest obstacle to comparison: no surveyed keyboard reports gaze error in degrees on held-out targets, and the one work that splits accuracy by axis reports a 17-point vertical deficit without explaining it. Third, the quantum machine-learning literature itself warns that fidelity kernels concentrate, that amplitude amplification helps only when the marked subspace is rare, and that no general advantage is known at small problem sizes. The proposed system is designed around these findings: it measures gaze continuously in degrees, attacks the vertical deficit at its cause, and admits a quantum component only where it beats a classical counterpart on held-out data.

CHAPTER 3. SYSTEM DESIGN

3.1 EXISTING SYSTEM

Overview

Two kinds of system exist for gaze-based text entry today. The first is the commercial infrared eye tracker paired with a dedicated communication application. An infrared illuminator creates corneal reflections, a camera tracks them together with the pupil, and a geometric eye model converts the two into a gaze direction with sub-degree accuracy. These systems work well, but their cost places them out of reach of most of the people who need them.

The second is the webcam gaze keyboard surveyed in Chapter 2. A typical pipeline detects the face with a Haar cascade, a HOG-and-SVM detector or a CNN; locates the eyes with the 68-point dlib landmark model; computes an eyelid-relative gaze ratio or an eye aspect ratio; and classifies the gaze into a small number of screen regions — often two to nine. Because the representation cannot support a continuous coordinate, keys are selected by blinking, by scanning through groups of keys, or by choosing among a few large regions. Head movement is ignored or forbidden, the thresholds are fixed by hand, and performance is reported as the percentage of frames assigned to the correct region.

Drawbacks of the Existing System

1. **Cost.** Infrared trackers reach sub-degree accuracy but need dedicated hardware that can cost more than the computer it attaches to.

2. **No pointing.** Systems built on the 68-point model have no iris landmarks, so they select keys by blinking or scanning among coarse regions, which is slow and tiring.
3. **Contaminated vertical feature.** The vertical iris feature is normalised by the eyelid aperture, which contracts with every squint, partial blink or change in lighting. Published systems report 93 % horizontal against only 76 % vertical accuracy, consistent with this defect.
4. **Head movement.** Features measured along image axes change when the head tilts; existing systems ignore head pose or prohibit movement.
5. **Uncorrected residual error.** A calibrated linear regression leaves a systematic, position-dependent error that nothing corrects frame by frame.
6. **Lossy key decision.** Gaze keyboards multiply a gaze probability by a language-model probability for each letter, which discards how the language evidence is distributed across candidate words, and they commit a key even when the evidence is weak.
7. **Incomparable evaluation.** Region-classification percentages over a handful of regions cannot tell a reader whether a user could point at an individual key.

3.2 PROPOSED SYSTEM

Overview

EYEQ is a hybrid quantum–classical gaze-typing system that runs entirely on a commodity webcam and an ordinary computer. It replaces the region-classification pipeline with continuous pointing at individual keys, and answers each drawback above with a specific mechanism: a commodity webcam instead of infrared

hardware; the MediaPipe iris ring and a ridge calibration for pointing; canthus normalisation against the contaminated vertical feature and against head roll; a quantum residual-correction register for the error the regression leaves behind; a gated fusion that asks instead of guessing, with QAOA re-decoding of finished words; and evaluation in degrees on held-out targets.

Features of the Proposed System

- **Estimation layer.** Landmarks are turned into a coarse gaze estimate through canthus normalisation and ridge regression, optionally refined by the quantum residual-correction stage.
- **Interaction layer.** The estimate is filtered, dwell is tracked with hysteresis, and the layer decides when a key has been selected.
- **Decision layer.** Gaze and language evidence are fused; the confidence gate and the movement gate decide whether to commit or ask; finished words are refined.

Design Principles

The critical path is sacred. A webcam delivers a frame every 33 ms, and the user experiences the dwell interval directly. Nothing quantum is permitted on the frame path or the dwell path: every quantum stage is trained offline, polled on its own timer, or run asynchronously at a word boundary.

Ask, never guess. A wrong letter costs a backspace and a second dwell; a prompt costs only one extra dwell. The decision layer therefore declines to commit below a confidence threshold and offers the leading candidates as chips.

Graceful degradation. Every server-side component is optional. If the quantum movement gate, the QAOA module or the research modules fail to load, the endpoint reports itself unavailable and the client falls back to its classical path. A watchdog disables any gate that refuses five consecutive keystrokes.

Measure before adopting. A component enters the typing path only if it beats the incumbent on held-out data, paired over participants. This rule removed one quantum stack from the typing path and kept another out of it.

Privacy by design. All gaze computation runs on the user's own device. The recorded evaluation sessions store landmark coordinates and screen geometry only — no video and no names.

3.3 SYSTEM ARCHITECTURE

Architectural Overview

Figure 5 shows the system block diagram with the reference numerals used in the patent invention disclosure.

The physical elements are few: a **commodity webcam (102)** capturing the face at about 30 frames per second, the **computing device (104)** running the browser client and a local server, and the **display (106)** showing the keyboard, calibration targets and suggestions. Everything else is software connected by data flow: the **landmark extractor (108)**; the **canthus normalisation and feature module (110)**; the **calibration and regression module (112)**, fitted on the **calibration targets (126)** and producing the coarse estimate G_c; the **residual correction module (114)** — error-state builder (116), correction-hypothesis register (118) and Born-rule readout (120) — whose correction ΔG a summing junction adds to give

$G^* = G_c + \Delta G$; the **filter and dwell module (122)**; and the **key decision module (124)**, which fuses the gaze likelihood with the **language model (128)** or, in the alternative embodiment, applies the quantum conditional selection rule.

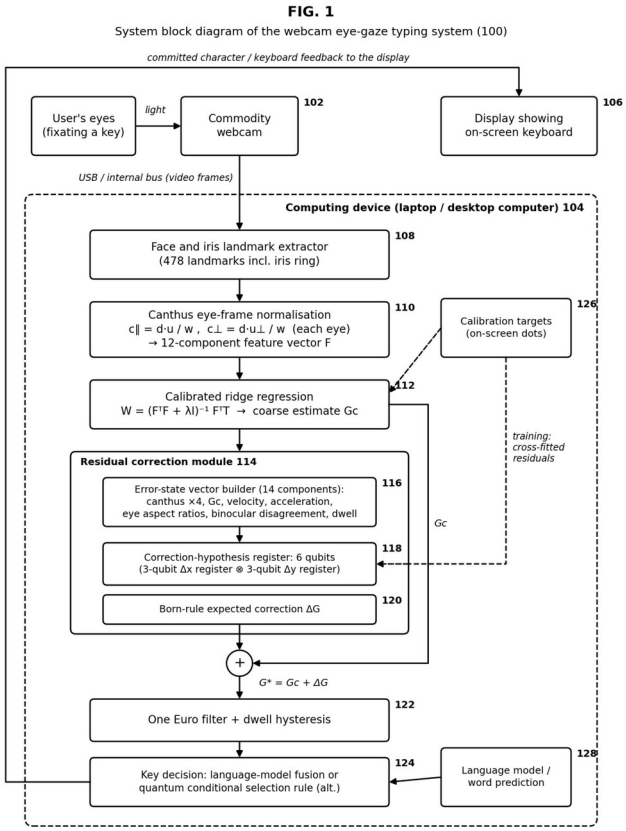


Figure 5 – System block diagram of the EYEQ webcam eye-gaze typing system (100)

Reference numerals follow the patent invention disclosure. The residual correction module (114) is shown in the position the invention places it; in the deployed prototype it is held out of the typing path pending confirmation against a size-matched classical model.

Architectural Flow

Figure 6 gives the per-frame process as a flowchart with steps S1–S12. The steps group into six stages.

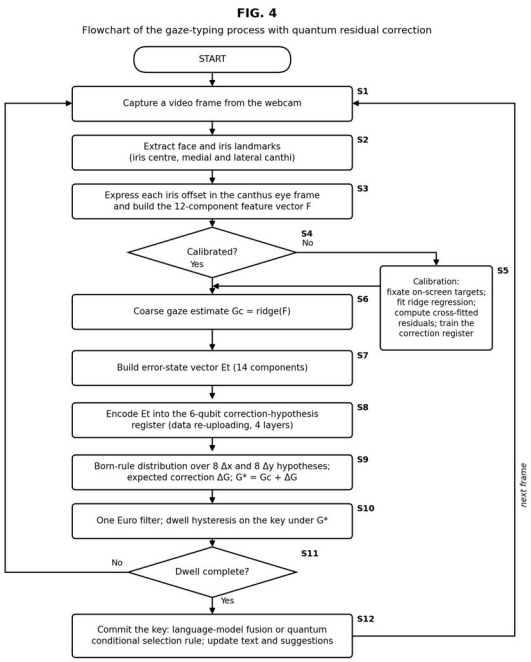


Figure 6 – Flowchart of the gaze-typing process, steps S1–S12, including calibration (S5) and the quantum residual correction stage (S7 –S9)

The loop repeats for every webcam frame.

Step 1: Input Acquisition

Each webcam frame (S1) is passed to the MediaPipe Face Landmarker, which returns 478 landmarks including the iris rings, and a facial transformation matrix (S2).

Step 2: Feature Construction

The iris offsets are expressed in the canthus eye frame and the twelve-component feature vector is assembled; blink frames are suppressed by an adaptive eye-aspect-ratio baseline (S3).

Step 3: Calibration

On first use the calibration branch shows nine targets anchored to key centres, fits the ridge regression and trains the correction register from cross-fitted residuals; the calibration is saved in the browser (S4–S5).

Step 4: Gaze Estimation

Every subsequent frame produces the coarse estimate (S6), the error-state vector and the register's expected correction (S7–S9).

Step 5: Filtering and Dwell

A One Euro-filtered estimate drives the dwell timer with hysteresis (S10–S11).

Step 6: Key Decision and Output

When the dwell completes, the key decision module either commits the character and refreshes the suggestions or offers choice chips; at a word boundary the finished word is re-decoded asynchronously (S12).

Module Description

Table 4 describes each software module, its input and output, the algorithm it uses and where it executes.

Table 4 – Software modules of the system

Module	Input	Output	Algorithm	Runs
Landmark extraction	Webcam frame	478 landmarks, transform matrix	MediaPipe Face Landmarker (GPU, else CPU/WASM)	Browser, every frame
Feature construction	Landmarks	12-component vector F	Canthus eye-frame normalisation	Browser, every frame

Module	Input	Output	Algorithm	Runs
Calibration	F at 9 targets × 5 clicks	Weight matrix W (12 × 2)	Ridge regression, Gauss–Jordan solve	Browser, once
Gaze mapping	F, W	Coarse estimate G_c and σ	Linear map with predictive variance	Browser, every frame
Residual correction	14-component error state	Correction ΔG	Six-qubit register, Born-rule expectation	Server (research)
Filtering and dwell	Gaze estimate	Dwell target, dwell centroid	One Euro filter, hysteresis	Browser, every frame
Key decision	Centroid, σ, prefix	Committed key or choice chips	Gaussian integral × language prior, gate θ	Browser, per dwell
Movement gate	Kinematic features	Usable / not usable, reliability	Quantum-kernel SVM, QNN → QBN	Server, polled 250 ms
Language model	Prefix	Letter prior, suggestions	Frequency trie + character 3-gram	Server, per letter
Word refinement	Per-letter evidence	Corrected word chips	QAOA re-decode ‖ noisy-channel correction	Server, on SPACE
Selection rule	Gaze, prior, hypotheses	Posterior over keys	Quantum conditional selection rule	Research module

Data Flow Diagram (DFD)

Level 0 (Context Diagram):

- User's gaze → EYEQ system → Committed text on screen

Level 1:

- Webcam frame → Landmark extraction → Feature construction → Gaze mapping → Filtering and dwell → Key decision → Text and suggestions
- Calibration targets → Calibration (ridge fit) → Weight matrix used by gaze mapping
- Typed prefix → Language model (server) → Letter prior used by key decision
- Eye kinematics → Movement gate and reliability (server, polled) → Commit or refuse

- Finished word → QAOA re-decode (server, asynchronous) → Correction chip

Latency Budget

Table 5 records the measured cost of each stage and confirms that no quantum stage sits on the critical path.

Table 5 – Latency budget

Stage	When it runs	Budget	Measured cost	On critical path?
Landmarks → features	Every frame	33 ms	Real time at camera rate (GPU)	Yes (classical)
Ridge refit	Once per calibration	—	0.017 ms	No
Ridge inference	Every frame	33 ms	0.42 μs per sample	Yes (classical)
Key posterior and fusion	Per dwell	693 ms	< 1 ms	Yes (classical)
QSVM movement gate	Polled every 250 ms	250 ms	9.8 ms	No — off-path poll
QBN reliability	Per poll	250 ms	< 1 ms, exact	No
QAOA word re-decode	On SPACE, asynchronous	—	≈ 6 ms	No — asynchronous
QAOA angle training	Offline, one time	—	minutes	No — offline

No quantum stage can extend the frame or dwell interval.

3.4 HARDWARE REQUIREMENTS

Minimum Hardware Configuration

Table 6 lists the minimum configuration; nothing beyond an ordinary computer and webcam is needed.

Table 6 – Minimum hardware configuration

Component	Specification	Purpose
Computer	Intel / AMD / Apple Silicon, 8 GB RAM or more	Runs the browser client and local server

Component	Specification	Purpose
Webcam	720p or 1080p, 30 fps, built-in or USB (about Rs. 1,900)	Face and iris capture
Display	Any; size and viewing distance recorded	Shows keyboard and targets
Infrared tracker	Not required	Replaced entirely by the webcam pipeline

Development and Evaluation Configuration

All development and every recorded session used the configuration in Table 7.

Table 7 – Hardware used for development and for the recorded evaluation sessions

Component	Specification
Processor and memory	Apple M4, 16 GB
Display	13-inch, 1470 × 839 CSS px, 29.4 × 19.1 cm
Viewing distance	40 cm (34.91 px per degree)
Operating system	macOS 15.6

3.5 SOFTWARE REQUIREMENTS

Table 8 lists the software on which the client, the server and the research modules depend.

Table 8 – Software requirements

Category	Component	Version / detail	Purpose
Operating system	macOS / Windows / Linux	Any with a modern browser	Host platform
Browser	Chromium-based or Safari	WebGL / WASM support	Keyboard client and camera access
Landmarks	MediaPipe Tasks Vision	0.10.14	Face Landmarker, 478 landmarks with iris ring
Client language	JavaScript (ES modules)	—	Tracking, calibration, filtering, dwell, fusion
Server language	Python	3.13	Language model, decoder and research modules
Web framework	FastAPI + Uvicorn	0.135 / 0.42	REST endpoints for suggestions and decisions
Numerical	NumPy, SciPy	2.2 / 1.17	Linear algebra, statistics

Category	Component	Version / detail	Purpose
Machine learning	scikit-learn	1.8	Classical baselines and SVMs
Quantum simulation	PennyLane	0.45	Quantum kernels, QSVM, QNN, QBN, QAOA
Deep learning	PyTorch, Transformers	2.8 / ≥ 4.46	Residual register training; cluster-to-text model
Testing	puppeteer-core, property checks	25.4	Headless end-to-end and correctness suites
Plotting	Matplotlib	3.10	Evaluation figures

Runtime Dependencies

The browser client loads the MediaPipe Tasks Vision 0.10.14 bundle and its WebAssembly runtime from the jsDelivr CDN, and the float16 Face Landmarker model from Google's model storage, when the page is opened. A network connection is therefore needed when the keyboard is first loaded, although every landmark and gaze computation then runs on the user's own device and no image leaves it. The server's dependencies are listed in `requirements.txt` (FastAPI, Uvicorn, Pydantic, NumPy, pandas, PyTorch, Transformers, Accelerate, Datasets, Evaluate and SacreBLEU); the quantum research modules additionally need PennyLane, SciPy and scikit-learn, which that file does not list, so they must be installed separately to reproduce the quantum results.

CHAPTER 4. IMPLEMENTATION

4.1 METHODOLOGY

The project followed a cycle of *build, instrument, record, measure, adopt* rather than a single design pass. Each component was first implemented in the simplest form that could be measured; the system was then instrumented so that every decision it made could be logged and replayed; real gaze was recorded from participants; and every candidate improvement was measured against the incumbent on the same recordings before it was allowed to change the typing path.

The adoption rule was fixed in advance and applied uniformly: **a component enters the typing path only if it beats the incumbent on held-out data, paired over participants, at $p < 0.05$.** For the quantum residual correction stage the rule was additionally *pre-registered* — written into the benchmark before any result was seen — so that it could not be bent after the fact. This rule is responsible for the shape of the final system as much as any algorithm: it removed one quantum stack from the typing path, kept another out of it, retained ridge regression over seven more sophisticated estimators, and admitted the canthus normalisation.

Project Timeline

Figure 7 shows the timeline of the work. The baseline keyboard was built during a summer internship at the National Institute of Technology Tiruchirappalli; the literature survey, methodology and architecture were completed for the first review; the quantum decision-layer experiments, the canthus normalisation and the participant recordings followed; and the work concluded with the IEEE conference manuscript, the quantum residual correction stage and the patent invention disclosure.

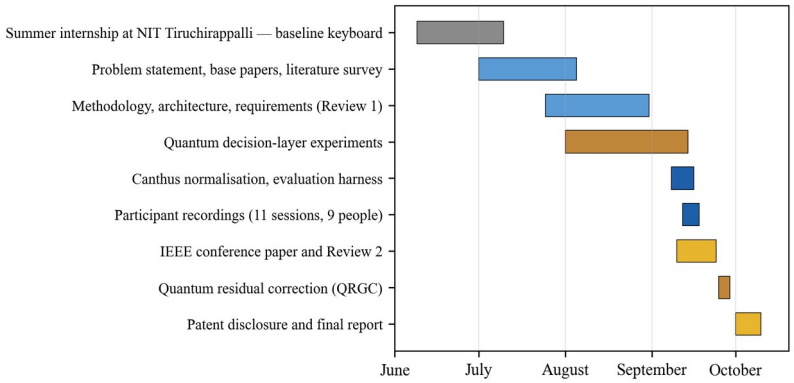


Figure 7 – Project timeline, June to October 2026

Evaluation Protocol

The protocol is described in full because the literature survey suggests that evaluation practice, rather than algorithmic choice, is the largest source of incomparability in this area.

Recording. Each session presents 21 on-screen targets: 13 used for fitting and 8 held out at positions the fit never sees, placed to span the corner, edge and centre regions of the display (Figure 8). Each target receives five fixations; for each fixation the first 150 ms are discarded to let the saccade settle and the next 300 ms of samples are retained. The participant's screen dimensions and viewing distance are recorded, so the conversion to degrees is measured rather than assumed.

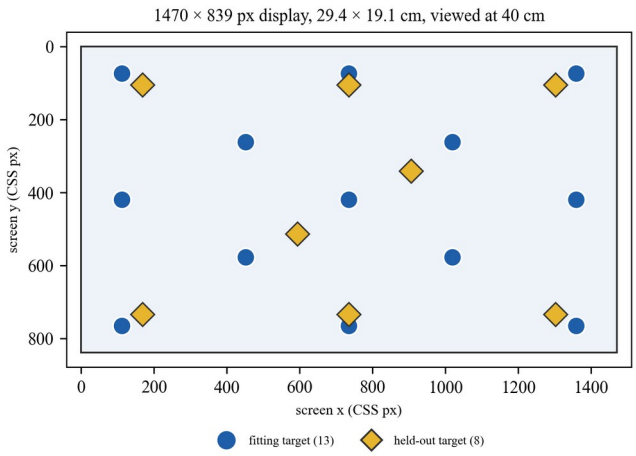


Figure 8 – The 21 target positions of an evaluation session on the recording display: 13 fitting targets and 8 held-out targets spanning corners, edges and centre

Every error reported in this report is measured on targets the fitted model did not see.

Splits. All splits are *grouped by target*. The five fixations recorded at one position are one observation measured five times, not five observations; splitting by sample leaves near-identical siblings in the training set and selects a regulariser far too small. Unless stated otherwise, results are leave-one-target-out (LOTO) over all 21 positions. Model-selection questions are additionally answered by leave-one-participant-out (LOPO), the question a system deployed on a new user actually faces.

Unit of analysis. Frames within a session are not independent, and neither are sessions from one person. Each participant therefore contributes the mean of their own sessions, and all significance tests are paired over participants.

Metrics. Errors are reported in *pixels* (what the dwell hit-test sees), *degrees of visual angle* (comparable with the gaze literature) and *fractions of a key pitch* (whether the error matters for typing). Decision rules are compared by commit accuracy, prompt rate, expected calibration error and entry rate.

Data quality. Sessions are admitted only if they cover the full target grid and lose less than 20 % of frames to tracking dropout; one session lost 67 % of its frames to poor illumination and is excluded, and its existence is reported rather than silently dropped. The admitted data are eleven sessions from nine participants (P01–P09). Table 9 summarises the protocol.

Table 9 – Summary of the evaluation protocol

Aspect	Setting
Participants / sessions	9 participants (P01–P09), 11 sessions
Display and distance	1470 × 839 CSS px, 29.4 × 19.1 cm, viewed at 40 cm (34.91 px per degree)
Targets per session	21 (13 fitting, 8 held out)
Fixations per target	5 × 300 ms retained, first 150 ms discarded
Primary split	Leave-one-target-out, grouped by target
Model selection	Leave-one-participant-out, model chosen inside each fold
Statistics	Paired t-test and Wilcoxon signed-rank over participants
Decision-rule simulation	16,839-word vocabulary; $\sigma \in \{30, 40, 50, 60, 75\}$ px against a 100 × 110 px key; identical seeded noise in both arms; matched prompt rate; 60-sentence training and 60-sentence held-out splits

Verification and Testing

Before any accuracy was measured, every algorithm was verified for *correctness*. An algorithm can run without raising an error and still be silently wrong — a distribution that does not sum to one, an encoding that differs from its classical oracle, a solution that is not actually optimal — and that kind of fault costs accuracy without ever appearing as a failure. The test suites therefore assert properties that must hold if each algorithm is implemented correctly, rather than merely checking that the code runs. Table 10 summarises them. The suites that need no browser were re-run for this report on 9 October 2026 and all passed; the five browser suites need Google Chrome, which was not installed on the machine used to prepare this report, so their status is the run of 28 September 2026 presented at the third review.

Table 10 – Verification and testing suites

Suite	What it verifies	Checks	Result
Correctness sweep `quantum/debug_all.py`	Quantum kernel symmetric, bounded, K(x, x) = 1; quantum Bayesian network equals classical enumeration (to 10^{-16}); QUBO-to-Ising round trip exact (to 10^{-15}); QAOA energy equals −log joint probability; ridge gradient zero at the solution (10^{-14}); metrics against hand computation	20	20 / 20 pass (re-run 9 Oct 2026)
Selection-rule properties `quantum/test_conditional_selection.py`	Equals log-linear fusion over 200 random inputs at zero phase; identical marginals separated by the rule; coherence in the unit disc; normalisation; amplification schedule	36	All hold (re-run 9 Oct 2026)
Residual-register checks `quantum/test_residual_correction.py`	Simulator equals PennyLane on random circuits; distributions sum to one; soft labels reproduce every in-range residual; the confidence gate never enlarges a correction	5	All pass (re-run 9 Oct 2026)
Gaze benchmark checks `eval/test_gaze_bench.py`	Synthetic sessions refused; grouped folds leak no target; pixels and degrees consistent; error grows with landmark noise (31.3 px → 151.0 px)	14	All pass (re-run 9 Oct 2026)
Evaluation layer and geometry `eval/test_commit_accuracy_log.py`, `eye_frame_check.js`, `pose_sign_check.js`	The commit logger leaves every model decision untouched; canthus features invariant to roll yet sensitive to gaze; every pose prompt matches the sign it implies	3 suites	All pass (re-run 9 Oct 2026)
End-to-end browser suites `tests/e2e/`	Calibration flow, dwell typing, movement-gate regression and watchdog, QAOA correction-chip flow, capture controls, in a headless browser	5 suites	Pass (run of 28 Sep 2026)

Each asserts properties that must hold if the algorithm is correct, not merely that the code runs.

4.2 IMPLEMENTATION

The implementation is organised into five modules, described in the order in which a webcam frame passes through them.

4.2.1 Landmark Acquisition and Canthus Feature Module

Landmark Acquisition

Landmarks are obtained from the MediaPipe Face Landmarker [16] (Tasks Vision 0.10.14), which runs in the browser on the GPU where available and falls back to CPU/WASM otherwise. For every frame it returns 478 normalised landmarks — the 468-point face mesh plus a five-point iris ring per eye — and a 4 × 4 facial transformation matrix. Figure 9 shows the landmarks the system uses.

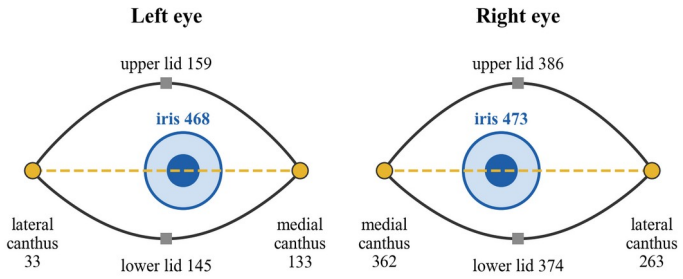


Figure 9 – Landmarks used by EYEQ

Each eye contributes an iris centre (468 left, 473 right), the lateral and medial canthi (33/133 left, 263/362 right) and the upper and lower lid points used for the conventional ratios and the eye aspect ratio. The dashed line is the inter-canthal baseline.

Two eye-aspect ratios derived from the lid and corner landmarks are used for blink rejection, and the nose landmark and inter-ocular width serve as head-translation and scale proxies.

Canthus-Relative Normalisation

The central idea of the estimation layer is to measure the iris against a structure that does not move with the eyelids. Let i, m and l denote the iris centre and the medial and lateral canthi of one eye. The eye axis and its perpendicular are

$$u = \frac{m - l}{\|m - l\|}, u^{\perp} = \begin{bmatrix} -u_y \\ u_x \end{bmatrix}, \tag{2}$$

and the iris offset is taken from the canthus midpoint, $d = i - 1/2(m + l)$. The two features per eye are the components of that offset in the eye's own frame, scaled by the inter-canthal distance $w = \|m - l\|$:

$$c_{\parallel} = \frac{d \cdot u}{w}, c_{\perp} = \frac{d \cdot u^{\perp}}{w}. \tag{3}$$

Figure 10 contrasts the two normalisations. Three properties follow from Equation (3), and each matters.

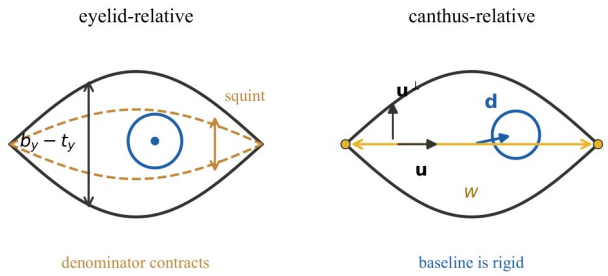


Figure 10 – Eyelid-relative against canthus-relative normalisation of the iris offset

Left: the conventional vertical feature divides by the eyelid aperture $b_y - t_y$, which contracts when the user squints. Right: the offset d is resolved along the eye axis u and its perpendicular $u^{\perp}$ and scaled by the inter-canthal distance w, a rigid baseline.

- **Independence from the eyelid.** Neither the offset nor the scale involves the lid landmarks, so squinting, partial blinks and lighting changes no longer move the vertical feature.
- **Scale stability.** The scale w is a Euclidean distance rather than a coordinate difference, so it does not collapse as the head turns away from the camera.

- **Roll invariance by construction.** Because u rotates with the head, both components are invariant to head roll. No roll angle is estimated, and none is needed. Measuring the offset along image axes instead mixes the channels by sin(roll) — about 3 % at 2° but 17 % at 10°, enough to inject a full key pitch of error.

Feature Vector

The complete feature vector concatenates the conventional representation with the four canthus components, as listed in Table 11. Adding the canthus components to the conventional eight is exactly the ablation reported in Section 5.4.

Table 11 – The 12-component feature vector used by the deployed gaze map

Index	Feature	Definition	Group
1	Bias	constant 1	Conventional
2–3	h_L, h_R	horizontal iris position between the eye corners	Conventional
4–5	v_L, v_R	vertical iris position between the eyelids (Equation (1))	Conventional
6–7	nose x, nose y	head-translation proxy	Conventional
8	face width	distance between lateral canthi, a scale proxy	Conventional
9–10	$c_{\parallel}^{L}$, $c_{\perp}^{L}$	left iris offset in the canthus frame (Equation (3))	Canthus
11–12	$c_{\parallel}^{R}$, $c_{\perp}^{R}$	right iris offset in the canthus frame	Canthus

Input Record

The system has two inputs: the webcam stream at about 30 frames per second, and, during calibration, the user's fixations on known targets. Both reach the algorithms as per-frame landmark records. Table 12 shows one such record exactly as stored by the evaluation capture tool — the first fixation sample of participant P01 at the top-left fitting target.

Table 12 – One recorded input sample (participant P01, first fixation)

Field	Value	Meaning
`t`	32146 ms	Time since the session started
`target_px`	(111.1, 73.2)	Target position on screen (CSS px)
`split`	fit	Used for fitting (as opposed to held out)
`f`	[1, 0.506, 0.601, 0.394, 0.427, 0.515, 0.837, 0.451]	Conventional 8-feature vector
`canthus_eye`	[0.0055, −0.1407, 0.0921, −0.1280]	$c_{\parallel}^{L}, c_{\perp}^{L}, c_{\parallel}^{R}, c_{\perp}^{R}$
`iris_r_l`, `iris_r_r`	0.0414, 0.0415	Iris-ring radius (normalised)
`ear_l`, `ear_r`	0.522, 0.517	Eye aspect ratios
`gaze_velocity`	0.0116	Normalised inter-frame motion
`quality_geometric`	1.0	Frame-quality weight
`yaw`, `pitch`, `roll`	0.011, 0.312, −0.022 rad	Head pose from the transform matrix

Each session also stores its screen geometry — 1470 × 839 CSS px on a 29.4 × 19.1 cm display viewed from 40 cm — so that pixel errors can be converted to degrees exactly. No video and no names are stored.

4.2.2 Calibration and Gaze Mapping Module

Ridge Calibration

Screen coordinates are obtained by ridge regression [35],

$$W = (F^{\top} F + \lambda I)^{-1} F^{\top} T, \tag{4}$$

where each row of F is a feature vector and each row of T the corresponding target in screen pixels. Equation (4) is the closed-form global minimiser of $\|FW - T\|^2 + \lambda \|W\|^2$; the objective is strictly convex, so the solution is unique and is attained exactly. The deployed model is a 12 × 2 weight matrix — 24 numbers — solved in the browser by Gauss–Jordan elimination with partial pivoting, with λ escalated automatically if the system is near-singular. The residuals also give a predictive variance σ^2, which the key decision uses as the width of the gaze likelihood.

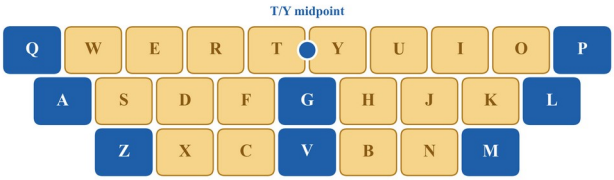


Figure 11 – The nine calibration targets, anchored to key centres: Q, P, A, G, L, Z, V, M and the midpoint of T and Y, since the top row has no middle key

Each target receives five fixation clicks.

Calibration uses nine on-screen targets anchored to key centres (Figure 11), spanning the corners, edges and centre of the keyboard. The user fixates each target and clicks it five times, which takes about a minute. Samples are weighted by a geometric frame-quality score that combines face scale, centring and landmark stability, and ordinary clicks gathered during later use are admitted as refinement samples at one-tenth of the weight of a calibration click, since they are not verified ground truth. The calibration is stored in the browser so that it is performed once per device.

Calibration Buffer and Frame Quality

Coverage-aware calibration eviction. The calibration buffer is bounded. Evicting the oldest samples first appears harmless and is not: it silently destroys spatial coverage, because refinement samples cluster wherever typing happens. A buffer was measured that had collapsed from thirteen occupied grid cells to two, with error rising from 4.93° to 8.25°. Eviction is therefore driven by grid occupancy, and the original calibration anchors are evicted last.

Geometric quality gating and an adaptive blink baseline. Frames are weighted by quality, and a per-user eye-aspect-ratio baseline — a high percentile of recent

frames — suppresses frames during blinks, because absolute eye aspect ratio varies more between people than a blink varies within one person. An early version passed the raw aspect ratio to a classifier trained on normalised openness, so every open eye was classified as a blink and the keyboard refused every keystroke; the fix and the watchdog are covered by regression tests.

User-centred pose convention. The signs of yaw and pitch from the facial transformation matrix are inverted so that they match how a user describes their own movement; this does not change accuracy, but it is a prerequisite for any posture-stratified analysis.

Dense Calibration and Re-centring (Experimental)

After the third review the calibration was extended on a separate experimental branch (`dense-calibration`), which is not part of the evaluated system. It walks a dot over all 28 keys with three fixations each, switches to kernel ridge once twelve or more distinct targets are available, and runs without clicks, so nothing is typed during calibration. Because a still-head calibration cannot learn what head position does to the map, the branch also adds a five-second, five-dot re-centre at known key positions (Q, P, G, Z and M), which measures the common offset and subtracts it from every estimate without refitting; it runs automatically when the head has stayed more than two calibration standard deviations from its calibrated position for three seconds. Its effect on the one recording available so far is reported in Section 5.4.

4.2.3 Filtering, Dwell and Key Decision Module

A coordinate emitted by a regressor is not yet a usable pointer. Several components between the two are load-bearing, and two of them were added in response to measured defects rather than intuition.

Temporal Filtering and Dwell

Temporal filtering. Predictions pass through a One Euro filter [36], whose cutoff frequency rises with speed so that a fixation is smoothed heavily while a saccade is not lagged. For each axis,

Dwell hysteresis. A cursor resting on a key boundary would otherwise oscillate between neighbours and commit neither. The dwell target is retained until the estimate leaves the key by 20 px for three consecutive frames, which converts boundary jitter into a stable selection. After a commit the same key is re-armed only after 450 ms, which allows deliberate double letters.

Gated Log-Linear Fusion

When the dwell completes, the system computes for each key k the probability that the gaze, modelled as an isotropic Gaussian centred on the dwell centroid μ with standard deviation σ, falls inside the key's rectangle $[x_0, x_1] \times [y_0, y_1]$:

$$P(k \mid \text{gaze}) \propto \left[\Phi\left(\frac{x_1 - \mu_x}{\sigma}\right) - \Phi\left(\frac{x_0 - \mu_x}{\sigma}\right) \right] \left[\Phi\left(\frac{y_1 - \mu_y}{\sigma}\right) - \Phi\left(\frac{y_0 - \mu_y}{\sigma}\right) \right], \tag{5}$$

where Φ is the standard normal cumulative distribution. This is an exact integral, not an approximation. The language prior $P(k \mid \text{prefix})$ comes from a frequency-weighted prefix trie over the 16,839-word vocabulary, mixed with a smoothed

character 3-gram so that out-of-vocabulary words remain typable. The two are fused log-linearly:

$$score(k) = P(k \mid \text{gaze}) \cdot max(P(k \mid \text{prefix}), \varepsilon)^{\beta}, \quad (6)$$

with $\beta = 0.70 < 1$ damping the prior so that the language model advises rather than vetoes, and $\varepsilon = 10^{-4}$ flooring it so that no letter becomes untypable. After normalisation, if the top candidate's probability is at least $\theta = 0.80$ the character is committed; otherwise the three leading candidates are shown as **choice chips**, and the user picks one with a single further dwell. Because a monotone recalibration cannot reorder candidates, Platt scaling [33] is applied to the stated confidence to make the threshold meaningful without changing which key wins (Section 5.4).

Reliability Gating

Reliability gating with a watchdog. A movement classifier refuses to commit from windows that look like saccades or blinks, and a Bayesian network converts its state estimate into $P(\text{key correct})$, which tempers the fused posterior (Section 4.2.4). A watchdog disables the gate after five consecutive refusals, since a gate whose input drifts out of distribution can otherwise refuse every keystroke without any visible error.

Language Model and Suggestions

The server maintains a frequency-weighted prefix trie over a 16,839-word English vocabulary and a smoothed character 3-gram model. The letter prior mixes the two (weight 0.85 on the trie) and is conditioned on the alphabet plus an end-of-word symbol, so its mass always sums to one. Word suggestions are the most frequent completions of the current prefix and are refreshed after every committed letter. When a finished word is not in the vocabulary, a gaze-aware noisy-channel

corrector proposes alternatives whose edit cost weights substitutions by QWERTY key distance, since a gaze error lands on a *neighbouring* key far more often than a distant one.

An alternative **cluster layout**, in which each key becomes a QWERTY region about three times wider than a letter key and words are resolved at the space by a decoder whose consistency oracle guarantees that every output re-encodes to the clusters gazed, was also implemented. On the pilot recordings it raised the fraction of fixations that cleared the threshold from 10.4 %, 2.9 % and 0.3 % to 52 %, 36 % and 5 %; the per-letter QWERTY keyboard was retained as the shipping layout, with the cluster layout preserved for Phase 2.

Parameters of the Deployed System

Table 13 lists the parameters of the deployed system.

Table 13 – Parameters of the deployed system

Parameter	Value	Role
Dwell time	693 ms	Fixation duration that commits a key
Re-arm interval	450 ms	Pause before the same key can fire again (double letters)
One Euro minimum cutoff f_{min}	0.30 Hz	Resting cutoff; lower is steadier
One Euro speed coefficient β	0.0006	How quickly the cutoff opens with speed
Dwell exit margin	20 px × 3 frames	Hysteresis before the dwell target switches
Calibration	9 targets × 5 clicks	Per-user ridge fit
Refinement weight	0.1	Weight of an unverified click relative to a calibration click
Gaze σ floor	14 px	Never trust the gaze below this standard deviation
Confidence threshold θ	0.80	Commit only above this fused confidence
Language exponent β	0.70	Damps the prior so gaze stays in charge
Language floor ε	10^{-4}	The language model may advise, never veto
Gate refusal limit	5	Consecutive refusals before the watchdog disables the gate

4.2.4 Quantum Decision Module

This module contains the quantum conditional selection rule — the decision-side formulation studied in this project — and the three quantum components of the hybrid decision layer.

What a Product of Marginals Discards

The second factor of Equation (6) is a *marginal*: the total frequency mass of every vocabulary word that continues the prefix with k. Forming that sum destroys the distinction between a letter supported by one dominant continuation and one supported by a diffuse spray of words that agree on nothing beyond a single character. After the prefix "th", the letters "e" and "r" may carry comparable marginal mass while meaning very different things, and Equation (6) cannot tell them apart. The quantum conditional selection rule was designed to keep that information. Figure 12 shows its stages.

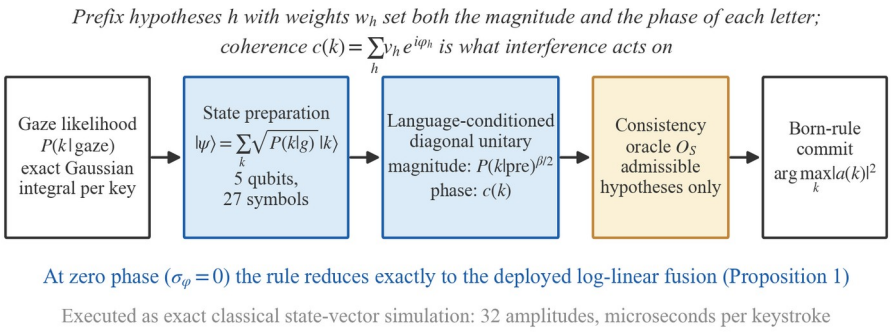


Figure 12 – Stages of the quantum conditional selection rule

Gaze likelihoods become amplitudes of a five-qubit register; a language-conditioned diagonal unitary attaches a magnitude and a phase to each candidate; a consistency oracle admits only hypotheses that re-encode to the gazed sequence; and the Born rule commits the most probable candidate.

The Rule

Let H be the vocabulary completions of the current prefix, each with frequency weight w_h and each proposing a next letter k_h. Candidates occupy a five-qubit register spanning *a–z* and a word-boundary symbol, prepared in superposition with amplitudes carrying the gaze likelihood:

$$|\psi\rangle = \sum_k \sqrt{P(k \mid \text{gaze})}\,|k\rangle. \tag{7}$$

A language-conditioned diagonal unitary then attaches to each candidate both a magnitude and a phase. Writing $v_h = w_h / P(k \mid \text{prefix})$ for the hypothesis weights *within* letter k, so that $\sum_h v_h = 1$, the conditioned amplitude is

$$a(k) = \sqrt{P(k \mid \text{gaze})}\,P(k \mid \text{prefix})^{\beta/2}\,c(k), \quad c(k) = \sum_{h:k_h=k} v_h e^{i\varphi_h}, \tag{8}$$

and the Born rule commits the most probable candidate, $P(k) = |a(k)|^2$. The factor $c(k)$ is the **coherence** of letter k, a weighted mean phasor with

$$|c(k)|^2 = \sum_h \sum_{h'} v_h v_{h'} \cos(\varphi_h - \varphi_{h'}), \tag{9}$$

whose off-diagonal terms are the interference. Since $|c(k)| \le 1$, with equality exactly when the supporting hypotheses agree in phase, coherence acts as a penalty on letters reachable only through mutually incoherent continuations. Phases are assigned from each hypothesis's mean per-character log-frequency, rescaled to $[0,1]$ across H and multiplied by a scale $\sigma_\varphi \pi$, so that the least surprising completion sits at phase zero. Length must enter the phase: phase driven by raw frequency alone would give a spray of comparably frequent completions one common phase, and they would interfere perfectly constructively in precisely the case the mechanism exists to catch.

Containment and Expressiveness

Proposition 1 (Containment). At $\sigma_\varphi = 0$, Equation (8) reduces to Equation (6).

Proof. With all $\varphi_h = 0$, $c(k) = \sum_h v_h = 1$ for every k, so $|a(k)|^2 = P(k \mid \text{gaze})\, P(k \mid \text{prefix})^\beta$, which is Equation (6) up to the shared normalisation. ▫

This is not a formality: a model class that cannot express the incumbent cannot fairly be said to beat it. The identity is asserted numerically to 10^{-12} over randomised inputs.

Proposition 2 (Expressiveness). No classical mixture over the same hypotheses reproduces Equation (9).

Proof sketch. A mixture assigns letter k the probability $\sum_h v_h^2$ after normalisation — the diagonal of Equation (9) alone. The off-diagonal terms depend on phase differences, and a mixture has no phase. Since $(\sum_h v_h)^2 > \sum_h v_h^2$ whenever more than one hypothesis contributes, the two disagree. ▫

Concretely, two hypothesis sets with *identical* marginals — and therefore indistinguishable to Equation (6) — receive different posteriors under Equation (8), and the diffuse set is the one penalised.

The Consistency Oracle and Amplitude Amplification

The correctness guarantee of the decoder is a predicate: a hypothesis is admissible only if re-encoding it reproduces exactly the sequence the user gazed. As an operator this is a projector O_S onto the admissible subspace, and it is what prevents the language model from ever proposing a word the gaze did not support. Given a

projector, amplitude amplification [25], the generalisation of Grover search [24] to a non-uniform initial distribution, rotates amplitude into the marked subspace using

$$Q = -(2|\psi\rangle\langle\psi| - I)O_S, m = \left\lfloor \frac{\pi/4}{\arcsin\sqrt{p_S}} \right\rfloor \tag{10}$$

rounds, where p_S is the marked mass. The intention was to let the top posterior clear the commit threshold from fewer frames of evidence and so shorten the dwell. Section 5.4 reports what it does instead.

Cost

Twenty-seven candidates occupy a 32-amplitude register, and preparation, conditioning, amplification and measurement cost microseconds. No quantum speed-up is claimed: the claim concerns the rule's *functional form*, not its execution, and the amplitude encoding is classically simulable by construction. The rule is implemented in `quantum/conditional_selection.py` as a research module with 36 property tests.

Hybrid Quantum–Classical Decision Layer

Three further quantum components were built for the decision layer, each with a defined input, output and classical counterpart.

Quantum-kernel movement gate (QSVM). The gate classifies the current eye state into fixation, saccade, blink or drift from four kinematic features — normalised eye openness, gaze velocity, fixation stability and a moving average of recent displacement. It is a support vector machine on a quantum fidelity kernel $K(x,z) = |\langle\phi(x)|\phi(z)\rangle|^2$ with an angle-encoding feature map on four qubits ($C = 16$). Prediction needs only its 24 support vectors and costs 9.8 ms, so the client polls it every 250 ms on its own timer rather than calling it at commit time. A keystroke

sampled from a window classified as a saccade or blink is refused and the user is asked to hold their gaze steady.

Reliability modelling (QNN → QBN). A five-node Bayesian network — blink, saccade, eye closed, unreliable signal, mis-key — encodes the causal story the tracker assumes: blinks and saccades corrupt the landmark signal, and a corrupted signal raises the risk of a wrong key. In the quantum Bayesian network each conditional probability becomes a controlled rotation $R_Y(2\arcsin\sqrt{p})$, so that measurement samples exactly from the joint distribution; it agrees with classical enumeration to 10^{-16}. Its output $P(\text{key correct})$ tempers the fused posterior as $p \leftarrow p^{\text{rel}}$, flattening it when the fixation looks suspect so that a marginal frame falls below the threshold and asks instead.

QAOA word re-decoding. Greedy commitment can never revise an early letter once a later one arrives. At each word boundary the system re-decodes the finished word jointly. Each typed position keeps its top two letter candidates with their fused probabilities; positions whose top probability is at least 0.90 are frozen, and each remaining run is a one-hot QUBO whose energy is

$$E(x)=\sum_{i,a} x_{i,a}\left(-\log P_i(a)\right)+\lambda\sum_{i}\sum_{a,b} x_{i,a}x_{i+1,b}\left(-\log P(b/a)\right), \tag{11}$$

so that the minimum-energy assignment is exactly the most probable word under the gaze evidence and letter-transition statistics. QAOA [31] with $p=2$ layers and an XY mixer [32], which keeps every measured state feasible, is run with angle schedules trained offline, so no optimiser runs at typing time. Every result is checked against exact enumeration of the chain, and any miss falls back to the exact optimum — the module can never be worse than greedy decoding. The call is

made asynchronously after the space key and returns in about 6 ms; its suggestion appears as a correction chip, so nothing is silently rewritten.

A removed stack. An earlier quantum gaze-to-letter stack — a variational classifier, a quantum-kernel SVM, quantum-kernel prototypes and a QAOA consensus — multiplied a third factor into every letter's score. Measured on 660 real recorded fixations it made typing slightly *worse* (26.8 % → 26.1 %), its own fusion discarded every quantum channel on 100 % of decisions, and it cost a median 68 ms per commit, so it was removed from the typing path under the adoption rule.

4.2.5 Quantum Residual Gaze Correction Module

Motivation

A calibrated linear regression leaves a systematic, position-dependent error — from optical curvature, eyeball-rotation nonlinearity and the momentary state of the eye — that nothing corrects frame by frame. The quantum residual gaze correction stage (QRGC) asks a narrower question than the ridge: *given the ridge's estimate and the current state of the eye, what error is the ridge making right now?* Its output ΔG gives the corrected estimate $G^{*} = G_c + \Delta G$.

Error-State Vector

Every frame, a 14-component error-state vector E_t is computed from quantities already available live (Table 14). Velocity and acceleration are finite differences of G_c within the current fixation and are reset at each new fixation; all components are standardised with calibration statistics.

Table 14 – The 14-component error-state vector read by the correction register

Components	Count	Meaning
Canthus offsets $c_{\parallel}^{L}, c_{\perp}^{L}, c_{\parallel}^{R}, c_{\perp}^{R}$	4	Where each iris sits in its eye frame
Coarse estimate $G_c = (\hat{x}, \hat{y})$	2	Where on the screen the ridge believes the user is looking
Velocity (v_x, v_y)	2	Finite difference of G_c within the fixation
Acceleration (a_x, a_y)	2	Second difference within the fixation
Eye aspect ratios (left, right)	2	Eye openness, which modulates landmark reliability
Binocular disagreement	1	How much the two eyes disagree about the gaze point
Time into the current fixation	1	Dwell elapsed, since early frames carry saccade residue

The Correction-Hypothesis Register

Six qubits form two three-qubit registers, one for Δx and one for Δy. Each basis state is one *correction hypothesis*: eight per axis, spaced 1.5° apart from −5.25° to +5.25°. The circuit (Figure 13) starts in $|000000\rangle$; in each of four layers, qubit q receives $R_Y(w \cdot E_{t,s} + b)$, where s cycles through the 14 components, followed by trainable R_Y and R_Z rotations and a CNOT ring $0 \to 1 \to \ldots \to 5 \to 0$ that entangles the two registers; a final trainable R_Y layer closes the circuit. Re-uploading the data in every layer increases expressivity without adding qubits [28]. The register has 102 trainable parameters and a 64-amplitude state.

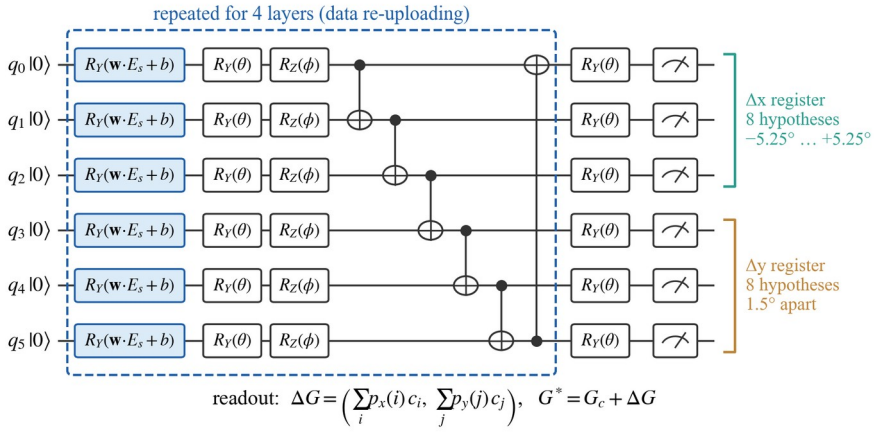


Figure 13 – The six-qubit correction-hypothesis register

Each layer re-uploads the error-state vector as rotation angles, applies trainable rotations and entangles the Δx and Δy registers with a CNOT ring; four layers are used. The Born distribution over each three-qubit register is read out as an expected correction.

Expected-Value Readout

The Born probabilities of the 64 basis states are marginalised onto qubits 0–2 (Δx) and 3–5 (Δy), and the correction is the *expectation* of each marginal rather than its most probable hypothesis:

$$\Delta G = \left(\sum_{i=0}^{7} p_x(i) c_i, \sum_{j=0}^{7} p_y(j) c_j \right), c_i = (i - 7/2) \times 1.5^{\circ}, \tag{12}$$

converted to pixels with the measured pixels-per-degree. Expected-value readout turns a discrete register into a continuous correction.

Cross-Fitted Training

The register is trained after calibration from the same calibration data, so it asks nothing extra of the user. Each calibration target's residual label comes from a ridge fitted *without* that target, so the register learns the error it will meet at unseen positions rather than the near-zero error of a target the ridge has already seen. Labels are split between the two nearest hypotheses so that the expectation of Equation (12) reproduces the residual exactly within range. Training minimises the cross-entropy against these soft labels with the Adam optimiser [37] (250 steps, learning rate 0.03). A gated variant scales each correction by the register's entropy-based confidence, $C = 1 - H(p)/\log 8$.

The register is simulated exactly on the CPU by a batched PyTorch state-vector simulator checked against PennyLane [30]; it uses only R_Y, R_Z and CNOT gates, but all results in this report come from exact classical simulation.

CHAPTER 5. CONCLUSION AND FUTURE WORK

5.1 CONCLUSION

Summary of the Work

This project built EYEQ, a gaze-typing system that runs entirely on a commodity webcam and an ordinary computer, and used it to study the two problems every webcam gaze keyboard must solve: estimating where the user is looking, and deciding which key they meant.

On the estimation side, the project identified a defect in a step the literature treats as settled: the conventional vertical iris feature is normalised by the eyelid aperture, which moves whenever the user squints, blinks or the lighting changes. Measuring the iris instead against the canthi, in the eye's own frame, removed that contamination and made the feature invariant to head roll by construction. Held-out gaze error fell from 5.55° to 3.76° (−32.3 %) for all nine participants, and the per-axis analysis showed the gain to be entirely vertical, exactly where the mechanism predicts. A six-qubit residual-correction register reduced the error a further 4.2 %, though not yet by a margin over an equally sized classical model that the adoption rule accepts.

On the decision side, the project formulated a quantum conditional selection rule, proved that it contains the conventional log-linear fusion exactly and expresses effects no classical mixture can, and showed by measurement that the extra expressiveness is worth nothing for English letter prediction, because the correct letter is the one with the most diverse linguistic support and therefore the lowest coherence. Around the rule, a hybrid decision layer was kept only where it earned its place: a quantum-kernel movement gate and a quantum Bayesian reliability

model that cut wrong keystrokes 38-fold in simulation, and a QAOA word re-decoder offered as correction chips.

Two short live typing trials, recorded for the third review, then tested whether these offline results carry over to real typing. They do not yet: commit accuracy was 18.2 % and 33.3 % against 95–99 % in simulation, and an analysis of a separate keyboard recording on an experimental branch identified posture changes after calibration, rather than the estimator, as the dominant error in live use.

The overarching lesson is that the accuracy came from the representation, not from the decision rule, and that quantum methods help where they add structure a classical method lacks, not where they replace a classical method that is already exact.

Objectives Revisited

Table 15 revisits each objective set out in Section 1.1.

Table 15 – Objectives and outcomes

Objective	Outcome	Status
1. Continuous gaze estimation on ordinary hardware	3.76° held-out error from a Rs. 1,900 webcam, reported in degrees on unseen targets	Achieved
2. Remove the dominant systematic error	Eyelid contamination identified; canthus normalisation −32.3 %, vertical error −53.2 %	Achieved
3. Reliable, fatigue-aware selection	Dwell with hysteresis and centroid; confidence gate with chips; Platt-calibrated confidence (ECE 0.0650 → 0.0167); live commit accuracy only 18–33 % in two short trials	Partly achieved
4. Investigate quantum decision algorithms rigorously	Selection rule with proofs; hybrid layer; residual register; every component compared with a classical counterpart	Achieved
5. Reproducible, honest evaluation	Nine participants, grouped splits, participant-level statistics, matched prompt rate, pre-registered adoption rule	Achieved

Objective	Outcome	Status
6. Real-time, private operation	All quantum stages off the critical path; all gaze computation on the device; no video stored	Achieved
7. Record negative results	Ten rejected ideas documented with their measurements and mechanisms	Achieved

Limitations

Sample size and population. The results rest on eleven sessions recorded from nine participants. This supports the principal claim — the canthus effect is large, improves every participant and survives a paired test by a wide margin — but it does not support a population estimate. Nine people recruited from one institution are not a sample of the population that would use an assistive keyboard, and the confidence interval of 1.35–2.23° describes the mean of this group rather than eye geometry in general. Four participants contributed two sessions and five contributed one, so between-session variance is estimated from four people.

A supported but not yet causal mechanism. The eyelid-contamination mechanism is supported by the ablation, by the per-axis analysis and by the asymmetry reported in the literature, but it has not been demonstrated causally. The decisive experiment perturbs eyelid aperture deliberately — by prompted squinting — while the gaze target is held fixed.

Live typing. The only live typing evidence is two short transcription trials by one participant, with 14 committed characters in total; commit accuracy was 18.2 % and 33.3 %. This is far too little data for a rate estimate, but far enough below the simulated 95–99 % to show that the offline results do not yet transfer to live use.

Simulation for the decision rules. The selection-rule and decision-layer results come from end-to-end simulation in which isotropic Gaussian noise stands in for the anisotropic, temporally correlated error of a real tracker. The negative result is

specific to one vocabulary and one language: the claim is that likelihood-derived phase is mis-signed for English letter prediction, not that interference-based fusion is useless in general.

Populations the system does not serve. Two exclusions are boundaries of applicability rather than shortfalls in accuracy, and they are stated because an assistive technology that overstates its reach does harm. The first is *oculomotor instability*: the method assumes a commanded fixation is held, so nystagmus, strabismus and saccadic intrusions break it by design. This is a pointed limitation, since motor-impaired users are more likely to present with these conditions, and a scanning or blink-driven interface remains the appropriate design for them. The second is *corrective eyewear*: lenses refract the apparent iris position, reflect under overhead light, and the frame can occlude the lateral canthus — the very landmark the normalisation is built on. Spectacle wearers were not recorded, and the accuracy reported here should not be assumed to hold for them.

One display and distance. All sessions were recorded at one display and one nominal viewing distance. The conversion to degrees is measured, but generalisation across geometries is untested. Large posture changes after calibration introduce a common offset that neither the ridge nor the residual register removes; on the one keyboard-protocol recording analysed, 7.6° of an 8.3° error was such an offset. The hands-free re-centre that addresses it exists only on an unmerged experimental branch.

Limitations of the quantum realisation. Every quantum algorithm in this project is executed as exact classical state-vector simulation; no quantum hardware is involved and no speed-up is claimed. The selection rule's register is small — superposition is over letters, not words — and its amplitude encoding is classically

simulable by construction. No component has been evaluated under gate noise, decoherence or finite sampling; since coherence is the first quantity a noisy device degrades, a null obtained in exact simulation is, if anything, optimistic.

Publications and Intellectual Property

Conference paper. A manuscript titled *"EYEQ: A Hybrid Quantum–Classical Eye-Gaze Typing System on Commodity Webcams"* by Devash Sivakumar, R. Anandan and V. Maruthi has been submitted to an IEEE conference and is under confidential peer review. It describes the canthus-relative normalisation and the quantum conditional selection rule.

Patent invention disclosure. An invention disclosure titled *"EYEQ: A Hybrid Quantum–Classical Eye-Gaze Typing System on Commodity Webcams"* has been prepared with the Registrar, VISTAS, as applicant and Devash Sivakumar, R. Anandan and V. Maruthi as inventors. It covers the canthus eye-frame representation, the two-stage estimation with a quantum correction-hypothesis register and Born-rule expected-value readout, and, as an alternative embodiment, the quantum conditional selection rule.

Concluding Remarks

This project set out to improve how a gaze keyboard decides, and improved how it estimates instead. The quantum conditional selection rule is a strict, provable generalisation of the fusion that gaze keyboards conventionally use, and its measured value is nil for a reason that no retuning addresses: phase derived from linguistic likelihood penalises exactly the evidence it should reward. Reporting that result honestly, with its mechanism, is a more durable contribution than an unreplicable gain would have been.

The accuracy came from the front end: four features computed from the canthi cut held-out error by a third, halved the vertical error, improved every participant and made a head-pose stage unnecessary, at no additional runtime cost. EYEQ shows that continuous gaze typing is achievable with a webcam costing a few hundred times less than an infrared tracker, and it provides a measured, reproducible foundation for the second phase, whose first task is to make that accuracy survive the posture changes of live use.

5.2 FUTURE WORK

The second phase of the project will pursue the following directions, ordered by the size of the expected effect.

1. **Posture-robust calibration.** Merge and evaluate the experimental 28-key hands-free calibration and the automatic five-dot re-centre, which reduced error from 8.74° to 4.70° and raised key hits from 4 % to 26 % on the single recording analysed so far, on recordings from several participants with deliberate posture changes.
2. **A live typing study.** Repeat the transcription protocol of Section 5.4 with several participants and longer phrase sets, so that live commit accuracy and entry rate can be reported with confidence intervals rather than from two short trials.
3. **The eyelid-perturbation experiment.** Record sessions with prompted squinting at fixed gaze targets. The prediction is specific: eyelid-relative features should degrade sharply and canthus-relative features should not. This converts the mechanism from a supported inference into a demonstrated one.

4. **Axis-specific feature sets.** Fit the vertical axis with the canthus components and the horizontal axis with the conventional ratios, to recover the 5.2 % horizontal cost identified in Section 5.4 while keeping the 53.2 % vertical gain.
5. **Confirmation of the residual register.** Run the pending kernel-ridge and support-vector residual controls, and test the ungated register against its classical twin on fresh recordings; adopt it into the typing path only if it passes the pre-registered rule.
6. **Broader recruitment and public datasets.** Extend the participant pool across eye geometries, ages and lighting, and evaluate canthus normalisation on established corpora — MPIIGaze [4], GazeCapture [5] and EYEDIAP [6]. Because the normalisation is computed from landmarks alone, the comparison is cheap to run.
7. **Spectacle wearers.** Quantify the accuracy gap for glasses, and establish whether per-user calibration absorbs the refractive offset or whether canthus occlusion is disqualifying — the most consequential gap in the present evaluation, since a large fraction of real users wear glasses.
8. **Constrained estimation.** Replace the unconstrained linear map with a physically constrained one — monotone in iris displacement, or an explicit eyeball-rotation model with the cornea centre as a latent parameter — so that prior geometry substitutes for calibration data. At nine targets ridge attains 4.80° and at twenty 3.63°, so roughly a quarter of the present error is calibration sparsity.
9. **Active optical alignment.** Mount the camera on a small pan–tilt platform driven by the head pose the pipeline already estimates, moving only between

fixations, to hold the iris near the optical axis where pixels per iris diameter are greatest.

The long-term target is a gaze error of 1.5°, falling to 1° — roughly a key width at typical seating distance — the point at which dwell selection on a full QWERTY layout becomes reliable enough that the language model advises rather than rescues.

5.3 SOURCE CODE

The source code is held in a private GitHub repository, `github.com/devash10-git/eyeq-gaze-keyboard`, created on 24 September 2026. On the `quantum-residual` branch from which this report was prepared, the repository tracks 208 files, comprising 15,987 lines of Python and 3,496 lines of JavaScript. Table 16 gives the layout of the principal files.

Table 16 – Repository layout

Path	Contents
`app.py`	FastAPI server: suggestions, letter prior, correction, decoder, movement gate, QAOA word re-decode
`static/iris-tracker.js`	Landmarks → canthus features → ridge calibration → screen coordinate
`static/app.js`	One Euro filter, dwell hysteresis, gated fusion, chips, movement-gate watchdog
`letters.py`, `decoder.py`, `clusters.py`	Letter prior, noisy-channel correction, cluster layout and word decoder
`quantum/conditional_selection.py`	The quantum conditional selection rule (research module)
`quantum/residual_correction.py`	The six-qubit quantum residual gaze-correction register
`quantum/movement_qsvm.py`, `qbn.py`, `typing_qaoa.py`	Movement gate, quantum Bayesian network, QAOA word re-decoding
`eval/`, `eval/sessions/`	Benchmarks, capture tool, figure scripts; eleven de-identified sessions from nine participants
`tests/e2e/`	Headless end-to-end browser suites

The keyboard is started with `python3 app.py` and opened at `http://127.0.0.1:8321`. Every number in Section 5.4 is produced by a script in `eval/` from the recorded sessions; nothing is copied by hand between tables.

```
pip install -r requirements.txt && python3 app.py   # run the keyboard
python3 eval/gaze_bench.py            # gaze accuracy in px, degrees and key
widths
python3 eval/feature_ablation.py      # the canthus result
python3 eval/lopo_validation.py       # leave-one-participant-out model selection
python3 eval/selection_rule_bench.py  # the quantum selection rule at matched
prompt rate
python3 eval/qrgc_bench.py            # the quantum residual correction stage
python3 -m quantum.debug_all          # correctness sweep over every algorithm
node tests/e2e/gate_regression.js     # headless end-to-end regression suites
```

The listings that follow are reproduced verbatim from the repository, with the file and line numbers of each, so that every excerpt can be checked against the source.

Listing 1 – Feature construction, including the canthus eye frame (static/iris-tracker.js, lines 602–660)

```
  _features(lms) {
    const L = LEFT_EYE, R = RIGHT_EYE, eps = 1e-6;
    const hL = (lms[L.iris].x - lms[L.outer].x) / (lms[L.inner].x - lms[L.outer].x
+ eps);
    const hR = (lms[R.iris].x - lms[R.inner].x) / (lms[R.outer].x - lms[R.inner].x
+ eps);
    const vL = (lms[L.iris].y - lms[L.top].y) / (lms[L.bottom].y - lms[L.top].y +
eps);
    const vR = (lms[R.iris].y - lms[R.top].y) / (lms[R.bottom].y - lms[R.top].y +
eps);
    const faceW = Math.hypot(lms[R.outer].x - lms[L.outer].x, lms[R.outer].y -
lms[L.outer].y);
    const base = [1, hL, hR, vL, vR, lms[NOSE].x, lms[NOSE].y, faceW];

    if (FEATURE_SET === "pose") {
      // Euler angles from the 4x4 facial transform, with a nose-offset fallback
      // when the matrix is unavailable. Interactions included because head
      // rotation changes what a given iris offset means, rather than shifting
      // the result by a constant.
      const m = this.lastMatrix;
      let yaw, pitch;
      // User-centred: yaw > 0 is a turn to the user's right, pitch > 0 is chin
      // up. MediaPipe's raw extraction gives the opposite for both — see the
      // note in eval/capture.html::poseFromMatrix.
      if (m && m.length >= 16) {
        yaw = -Math.atan2(m[8], m[10]);
        pitch = -Math.atan2(-m[9], Math.hypot(m[8], m[10]));
      } else {
        yaw = 0.5 - lms[NOSE].x;
        pitch = 0.5 - lms[NOSE].y;
      }
      const h = (hL + hR) / 2, v = (vL + vR) / 2;
```

```
    return base.concat([yaw, pitch, h * yaw, v * pitch, h * pitch, v * yaw]);
  }
  if (FEATURE_SET !== "canthus") return base;

  /* Iris offset from the corner midpoint, expressed in the EYE's own frame.
   *
   * The earlier version divided by the 2-D corner distance — correct, so the
   * scale did not shrink as the head turned — but then took the offset's
   * image-space x and y. Those are horizontal and vertical *in the camera
   * frame*, not along the eye. Tilt the head and the eye axis rotates under
   * them, so a purely sideways glance starts producing vertical signal.
   *
   * Projecting onto the canthus axis and its perpendicular removes that by
   * construction: both components are invariant to head roll, with no roll
   * angle needed as an input. The cost of not doing it grows as sin(roll) —
   * about 3% cross-mixing at 2 degrees, which is why it was invisible in the
   * recordings, but 17% at 10 degrees, which injects a full key pitch. */
  const eyeFrame = (E) => {
    const inner = lms[E.inner], outer = lms[E.outer], iris = lms[E.iris];
    // Axis chosen per eye so both features increase in the same direction,
    // matching the convention hL/hR already use.
    const ax = (E === LEFT_EYE ? inner.x - outer.x : outer.x - inner.x);
    const ay = (E === LEFT_EYE ? inner.y - outer.y : outer.y - inner.y);
    const w = Math.hypot(ax, ay) + eps;
    const ux = ax / w, uy = ay / w;                 // unit vector along the eye
    const dx = iris.x - (inner.x + outer.x) / 2;
    const dy = iris.y - (inner.y + outer.y) / 2;
    return [(dx * ux + dy * uy) / w,                // along the eye axis
            (-dx * uy + dy * ux) / w];              // perpendicular to it
  };
  return base.concat([...eyeFrame(L), ...eyeFrame(R)]);
}
```

Listing 2 – Ridge calibration with an escalating regulariser (static/iris-tracker.js, lines 946–977)

```
_fit() {
  const d = FEAT_DIM;
  const A0 = Array.from({ length: d }, () => new Array(d).fill(0));
  const bx = new Array(d).fill(0), by = new Array(d).fill(0);
  for (const s of this.samples) {
    const w = s.w == null ? 1 : s.w; // older saved samples carry no weight
    for (let i = 0; i < d; i++) {
      bx[i] += w * s.f[i] * s.x;
      by[i] += w * s.f[i] * s.y;
      for (let j = 0; j < d; j++) A0[i][j] += w * s.f[i] * s.f[j];
    }
  }
  // ridge scaled to the matrix, escalating until the system solves cleanly
  // (correlated eye features can make the unregularized system singular)
  const meanDiag = A0.reduce((sum, row, i) => sum + row[i], 0) / d;
  let lam = 1e-4 * meanDiag;
  for (let tries = 0; tries < 6; tries++) {
    const A = A0.map((row, i) => row.map((v, j) => (i === j ? v + lam : v)));
    const wx = solveLinear(A, bx);
    const wy = solveLinear(A, by);
    if (wx && wy) {
      this.wx = wx; this.wy = wy;
      this._finalizeFit(A);
```

```
        // The kernel model is an addition, never a replacement: the linear
        // solution above stays live as the fallback and as the source of the
        // per-prediction sigma the commit gate reads.
        this.krr = this.model === "krr" ? this._krrFit() : null;
        return;
      }
      lam *= 10;
    }
  }
```

Listing 3 – One Euro filter (static/app.js, lines 497–512)

```
class OneEuroAxis {
  constructor() { this.x = null; this.dx = 0; this.t = null; }
  _alpha(cutoff, dt) { const tau = 1 / (2 * Math.PI * cutoff); return 1 / (1 + tau
/ dt); }
  filter(x, t) {
    if (this.x === null) { this.x = x; this.t = t; return x; }
    const dt = Math.max(1e-3, (t - this.t) / 1000);
    this.t = t;
    const raw = (x - this.x) / dt;
    this.dx += this._alpha(EURO_DCUTOFF, dt) * (raw - this.dx);
    const cutoff = EURO_MIN_CUTOFF + EURO_BETA * Math.abs(this.dx);
    this.x += this._alpha(cutoff, dt) * (x - this.x);
    return this.x;
  }
  reset() { this.x = null; this.dx = 0; this.t = null; }
}
const euroX = new OneEuroAxis(), euroY = new OneEuroAxis();
```

Listing 4 – Dwell tracking with hysteresis and the dwell centroid (static/app.js, lines 554–603)

```
function updateDwell() {
  const now = performance.now();
  let key = keyAt(point.x, point.y);

  // Stickiness: a dwell in progress survives a pointer that strays briefly or
  // barely. Only a departure that is both beyond the margin and sustained for
  // several frames actually changes target.
  if (key !== dwellKey && dwellKey) {
    if (withinMargin(dwellKey, point, DWELL_EXIT_MARGIN)) {
      outsideFrames = 0;
      key = dwellKey;
    } else if (++outsideFrames < DWELL_EXIT_FRAMES) {
      key = dwellKey;
    }
  } else {
    outsideFrames = 0;
  }

  keys.forEach(k => k.classList.toggle("active", k === key));
  chipEls.forEach(c => c.classList.toggle("active", c === key));

  if (key !== dwellKey) {
    dwellKey = key;
    dwellStart = now;
    dwellPts = [];
    outsideFrames = 0;
```

```
    ring.classList.remove("filling");
    void ring.offsetWidth; // restart the fill animation
    if (key && now >= rearmUntil) ring.classList.add("filling");
    return;
  }
  if (!key || now < rearmUntil) return;
  dwellPts.push({ x: point.x, y: point.y });
  if (dwellPts.length > 60) dwellPts.shift();

  if (!ring.classList.contains("filling")) {
    dwellStart = now;
    ring.classList.add("filling");
  }
  if (now - dwellStart >= DWELL_MS) {
    // chips commit directly; physical keys go through the fused posterior,
    // judged on the whole dwell window rather than its last frame
    if (key.classList.contains("chip")) commit(key);
    else smartCommit(key, dwellCentroid());
    rearmUntil = now + REARM_MS;
    dwellStart = now;
    dwellPts = [];
    ring.classList.remove("filling");
  }
}
```

Listing 5 – Gaze posterior over keys (static/app.js, lines 233–250)

```
/* P(key | gaze): integral of the gaze Gaussian over each key rectangle
 * (exact for an axis-aligned Gaussian: product of two 1-D CDF differences). */
function keyPosterior(mu, sig) {
  const out = [];
  let total = 0;
  for (const k of keys) {
    const r = k.getBoundingClientRect();
    const px = (normCdf(r.right, mu.x, sig.sx) - normCdf(r.left, mu.x, sig.sx)) *
               (normCdf(r.bottom, mu.y, sig.sy) - normCdf(r.top, mu.y, sig.sy));
    if (px > 1e-6) {
      out.push({ el: k, letter: k.dataset.letter || null, action: k.dataset.action
|| null, pGaze: px });
      total += px;
    }
  }
  if (total <= 1e-9) return [];
  out.forEach(c => c.pGaze /= total);
  return out.sort((a, b) => b.pGaze - a.pGaze);
}
```

Listing 6 – Commit or ask (static/app.js, lines 364–381)

```
/* The commit tail of smartCommit, split out so a test session can supply the
 * decision without duplicating what happens once one is made. */
function applyDecision(top, fused) {
  if (top.p >= THETA) {
    top.el.classList.add("flash");
    setTimeout(() => top.el.classList.remove("flash"), 200);
    if (top.letter) {
      // record the letters-only posterior for this position so the whole word
      // can be jointly re-decoded (QAOA) at the word boundary
```

```
      const letters = fused.filter(c => c.letter).slice(0, 2);
      const z2 = letters.reduce((s, c) => s + c.p, 0) || 1;
      applyLetter(top.letter, letters.map(c => [c.letter, +(c.p /
z2).toFixed(4)]));
    } else {
      applySpace();
    }
    return;
  }
```

Listing 7 – Preparation, coherence and conditioning of the selection rule (quantum/conditional_selection.py, lines 107–128)

```
def prepare(p_gaze: Dict[str, float]) -> np.ndarray:
    """Amplitude-encode the gaze likelihood: ``|a> = sum_k sqrt(P(k|gaze)) |k>``.

    Args:
        p_gaze: ``{letter: P(letter | gaze)}``. Need not be normalised; the
            returned state always is. Letters outside ``ALPHABET`` are ignored.

    Returns:
        A complex state vector of length ``DIM`` with unit norm. An all-zero or
        empty input returns the uniform state over the alphabet, which is the
        honest representation of "the gaze tells us nothing".
    """
    amp = np.zeros(DIM, dtype=complex)
    for ch, p in p_gaze.items():
        i = INDEX.get(ch)
        if i is not None and p > 0:
            amp[i] = math.sqrt(p)
    n = np.linalg.norm(amp)
    if n < 1e-12:
        amp[:len(ALPHABET)] = 1.0 / math.sqrt(len(ALPHABET))
        return amp
    return amp / n
```

Listing 8 – The correction-hypothesis register (quantum/residual_correction.py, lines 115–175)

```
class CorrectionRegister(torch.nn.Module):
    """Data re-uploading circuit whose measurement is a distribution over
corrections."""

    def __init__(self, d: int, layers: int = 4, seed: int = 0):
        super().__init__()
        n = 2 * N_REG
        self.sim, self.n, self.layers = Sim(n), n, layers
        g = torch.Generator().manual_seed(seed)
        self.w = torch.nn.Parameter(torch.ones(layers, n))
        self.b = torch.nn.Parameter(torch.zeros(layers, n))
        self.rot = torch.nn.Parameter(0.1 * torch.randn(layers, n, 2,
generator=g))
        self.fin = torch.nn.Parameter(0.1 * torch.randn(n, generator=g))
        # which feature each (layer, qubit) slot re-uploads: all d features are
        # visited, cycling, so every feature reaches both registers
        self.register_buffer("slot", torch.tensor([[(l * n + q) % d for q in
range(n)]
                                                   for l in range(layers)]))
```

```
    def _fixed_layer(self, l: int) -> torch.Tensor:
        """The data-independent part of layer l as one matrix M (psi_out = psi_in
@ M).

        Built by running the identity through the exact gate sequence, so it is
        the same circuit as ``forward_gates`` by construction (and tested).
        """
        s, psi = self.sim, torch.eye(self.sim.N, dtype=torch.complex128)
        for q in range(self.n):
            psi = s.ry(psi, q, self.rot[l, q, 0])
            psi = s.rz(psi, q, self.rot[l, q, 1])
        for q in range(self.n):
            psi = s.cnot(psi, q, (q + 1) % self.n)
        return psi

    def _final(self) -> torch.Tensor:
        s, psi = self.sim, torch.eye(self.sim.N, dtype=torch.complex128)
        for q in range(self.n):
            psi = s.ry(psi, q, self.fin[q])
        return psi

    def forward(self, x: torch.Tensor) -> Tuple[torch.Tensor, torch.Tensor]:
        s, psi = self.sim, self.sim.zero(x.shape[0])
        for l in range(self.layers):
            for q in range(self.n):
                psi = s.ry(psi, q, self.w[l, q] * x[:, self.slot[l, q]] +
self.b[l, q])
            psi = psi @ self._fixed_layer(l)
        psi = psi @ self._final()
        p = (psi.abs() ** 2).reshape(-1, K, K)            # Born rule; qubits 0-2
= dx, 3-5 = dy
        return p.sum(2), p.sum(1)

    def forward_gates(self, x: torch.Tensor) -> Tuple[torch.Tensor, torch.Tensor]:
        """Reference: the same circuit applied one gate at a time."""
        s, psi = self.sim, self.sim.zero(x.shape[0])
        for l in range(self.layers):
            for q in range(self.n):
                psi = s.ry(psi, q, self.w[l, q] * x[:, self.slot[l, q]] +
self.b[l, q])
                psi = s.ry(psi, q, self.rot[l, q, 0])
                psi = s.rz(psi, q, self.rot[l, q, 1])
            for q in range(self.n):
                psi = s.cnot(psi, q, (q + 1) % self.n)
        for q in range(self.n):
            psi = s.ry(psi, q, self.fin[q])
        p = (psi.abs() ** 2).reshape(-1, K, K)
        return p.sum(2), p.sum(1)
```

Listing 9 – Soft labels, readout and training (quantum/residual_correction.py, lines 200–239)

```
def soft_labels(r_deg: np.ndarray) -> np.ndarray:
    """Split each residual between its two nearest hypotheses so E[c] = r (inside
range)."""
    r = np.clip(r_deg, CENTRES_DEG[0], CENTRES_DEG[-1])
    pos = (r - CENTRES_DEG[0]) / SPACING_DEG
    lo = np.clip(np.floor(pos).astype(int), 0, K - 2)
```

```
    frac = pos - lo
    out = np.zeros((len(r), K))
    out[np.arange(len(r)), lo] = 1 - frac
    out[np.arange(len(r)), lo + 1] = frac
    return out


def readout(px: np.ndarray, py: np.ndarray, gate: bool = True) -> np.ndarray:
    """Expected correction in degrees, optionally scaled by the register's own
confidence."""
    c = CENTRES_DEG
    d = np.column_stack([px @ c, py @ c])
    if not gate:
        return d
    ent = lambda p: -(p * np.log(np.clip(p, 1e-12, 1))).sum(1) / math.log(K)
    conf = np.column_stack([1 - ent(px), 1 - ent(py)])
    return conf * d


def fit_register(model: torch.nn.Module, Z: np.ndarray, R_deg: np.ndarray,
                 steps: int = 250, lr: float = 0.03) -> Callable[[np.ndarray],
Tuple[np.ndarray, np.ndarray]]:
    X = torch.tensor(Z)
    Lx, Ly = torch.tensor(soft_labels(R_deg[:, 0])),
torch.tensor(soft_labels(R_deg[:, 1]))
    opt = torch.optim.Adam(model.parameters(), lr=lr)
    for _ in range(steps):
        opt.zero_grad()
        px, py = model(X)
        loss = -(Lx * torch.log(px + 1e-12)).sum(1).mean() - (Ly * torch.log(py +
1e-12)).sum(1).mean()
        loss.backward()
        opt.step()

    def predict(Q):
        with torch.no_grad():
            px, py = model(torch.tensor(Q))
        return px.numpy(), py.numpy()
    return predict
```

Listing 10 – Quantum Bayesian network for keystroke reliability (quantum/qbn.py, lines 82–95)

```
class QuantumBayesianNetwork:
    """Exact quantum encoding of a discrete Bayesian network.

    One qubit per node; CPTs become (multi-)controlled RY rotations. The
    prepared state's basis-measurement distribution equals the network's joint
    distribution, which is asserted against classical enumeration in the tests.
    """

    def __init__(self, nodes: Optional[Sequence[Node]] = None) -> None:
        self.nodes: List[Node] = list(nodes) if nodes else default_gaze_network()
        self.index = {n.name: i for i, n in enumerate(self.nodes)}
        self.n_qubits = len(self.nodes)
        self.dev = qml.device("default.qubit", wires=self.n_qubits)
        self._probs = qml.QNode(self._circuit, self.dev)
```

Listing 11 – QUBO for a word segment and its exact chain solution (quantum/typing_qaoa.py, lines 110–169)

```
def build_segment_qubo(
    cands: Sequence[Sequence[Tuple[str, float]]],
    stats: LetterStats,
    left: Optional[str],
    right: Optional[str],
    lam: float = LAMBDA,
) -> np.ndarray:
    """QUBO over one run of ambiguous positions.

    Args:
        cands: per position, ``K`` ``(letter, gaze_prob)`` candidates.
        stats: letter statistics.
        left/right: fixed letters adjacent to the segment (or ``None`` at the
            word edge). Their transition terms fold into the unary diagonal —
            this is how frozen positions constrain the segment without qubits.
    """
    n, k = len(cands), len(cands[0])
    N = n * k
    Q = np.zeros((N, N))

    for i, pos in enumerate(cands):
        for a, (ch, p) in enumerate(pos):
            u = -math.log(max(p, _EPS))                          # gaze evidence
            if i == 0:
                u += lam * -math.log(
                    stats.p_trans(left, ch) if left else stats.p_start(ch))
            if i == n - 1 and right:
                u += lam * -math.log(stats.p_trans(ch, right))
            Q[i * k + a, i * k + a] += u

    for i in range(n - 1):                                       # chain coupling
        for a, (c1, _p1) in enumerate(cands[i]):
            for b, (c2, _p2) in enumerate(cands[i + 1]):
                Q[i * k + a, (i + 1) * k + b] += lam * -math.log(
                    max(stats.p_trans(c1, c2), _EPS))
    return Q


def chain_exact(
    cands: Sequence[Sequence[Tuple[str, float]]],
    stats: LetterStats,
    left: Optional[str],
    right: Optional[str],
    lam: float = LAMBDA,
) -> Tuple[List[int], float]:
    """Exact MAP over the segment by enumeration (K^n <= 64 — microseconds).

    The classical yardstick and the fallback guarantee.
    """
    n = len(cands)
    Q = build_segment_qubo(cands, stats, left, right, lam)
    best, best_e = None, math.inf
    for combo in iproduct(range(K), repeat=n):
        x = np.zeros(n * K)
        for i, a in enumerate(combo):
            x[i * K + a] = 1.0
```

```
        e = qubo_energy(Q, x)
        if e < best_e:
            best_e, best = e, list(combo)
    return best, float(best_e)
```

5.4 RESULTS

This section reports the measured results of the project. It begins with sample outputs captured from the running system and then gives the quantitative results, all computed by the scripts in `eval/` from the recorded sessions or from seeded simulation, as stated for each.

Sample Outputs

The output of the system is committed text, together with the interface elements that let the user steer it. The three characteristic outputs below were captured from the running system: word suggestions (Figure 14), a QAOA correction chip (Figure 15) and choice chips offered instead of a guess (Figure 16).



Figure 14 – Output 1 — live word suggestions

After the letters “wor” are committed, the most frequent completions are offered as chips above the keyboard and can be selected with a single dwell.

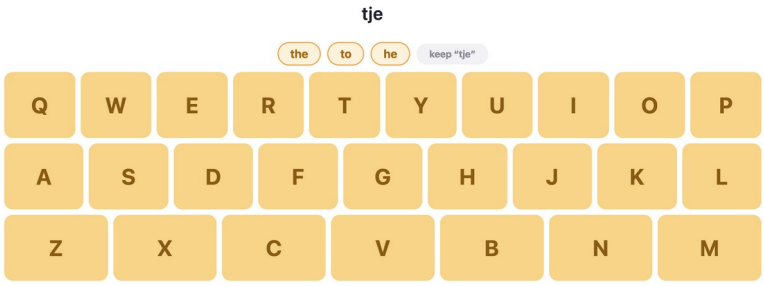


Figure 15 – Output 2 — QAOA word re-decoding

The gaze wobbled on the second letter, so greedy commitment produced “tje”. When the space key is dwelt, QAOA re-decodes the word jointly from the per-letter evidence and offers “the” as the first correction chip (6 qubits, matched the exact optimum, about 6 ms). The user may also keep the word as typed.

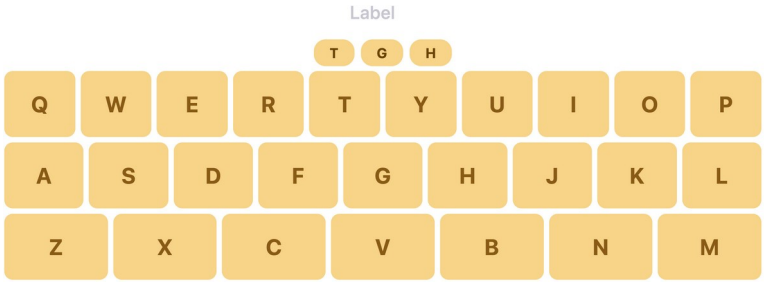


Figure 16 – Output 3 — ask, never guess

A wide gaze scatter near T, G and H leaves no candidate above the confidence threshold, so instead of committing a possibly wrong key the system offers the three leading candidates as choice chips.

Feature Ablation

Table 17 gives the principal result of the project. Canthus normalisation reduces held-out error from $5.552 \pm 0.488^{\circ}$ to $3.762 \pm 0.438^{\circ}$, a mean reduction of 1.791° or 32.3 %. The comparison is paired within participant: $t(8) = 9.31$, $p = 1.4 \times 10^{-5}$; Wilcoxon $W = 0$, $p = 0.0039$; 95 % confidence interval on the reduction $[1.35, 2.23]^{\circ}$; effect size $d_z = 3.10$. **Every one of the nine participants improved**, by between 19.5 % and 44.6 %.

Table 17 – Feature ablation, leave-one-target-out over 21 positions, eleven sessions from nine participants (mean ± standard deviation over participants)

Feature set	Dim	Error (px)	Error (deg)	vs deployed
Deployed (eyelid-relative)	8	193.8	5.552 ± 0.488	baseline
+ iris-ring radius	11	193.0	5.529 ± 0.488	+0.4 %
+ head pose only	14	182.7	5.233 ± 0.337	+5.7 %
+ canthus (proposed)	**12**	**131.3**	**3.762 ± 0.438**	**+32.3 %**
+ canthus + radius	15	131.5	3.767 ± 0.443	+32.1 %
+ canthus + pose	21	132.2	3.788 ± 0.493	+31.8 %
+ all of the above	24	127.9	3.663 ± 0.474	+34.0 %

Lower error is better.

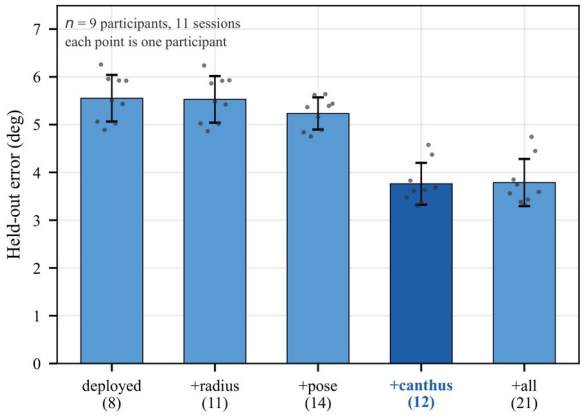


Figure 17 – Held-out error by feature set over the nine participants

Bars are the mean across participants and whiskers one standard deviation; each participant contributes the mean of their own sessions, and all nine are overlaid as points so the spread is visible rather than summarised.

Three rows of the table isolate the cause (Figure 17). An *iris-ring radius* scale cue yields 0.4 %, within noise, so the gain is attributable to the canthi and not merely to more features. *Head pose alone* is worth 5.7 %, the size of the effect the canthus frame subsumes; added on top of the canthus frame it yields nothing detectable (31.8 % against 32.3 %, $p = 0.439$), because the canthus frame is already attached to the head. Canthus normalisation is thus the *minimal sufficient* addition. The 24-dimensional set is marginally lower still, but not significantly, and it doubles the calibration burden, so the 12-component vector is deployed.

Per-Participant Analysis

Figure 18 draws the same comparison participant by participant. The improvement is not carried by a few favourable individuals: every line slopes downward, the smallest reduction is 19.5 % and the largest 44.6 %. The two participants with the largest remaining error under canthus features, P07 (4.58°) and P08 (4.37°), are also the two who improved least (22.7 % and 19.5 %); their residual error is the natural target for the residual-correction stage of Section 5.4.

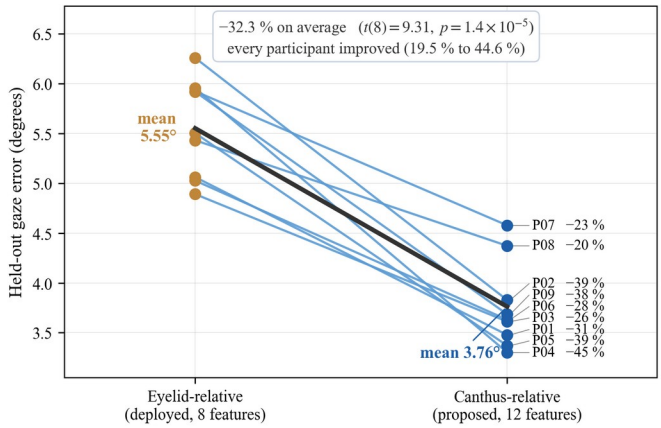


Figure 18 – Held-out error per participant under the deployed and the proposed features

Each line is one participant; the heavy line is the mean. Every participant improved.

Figure 19 shows what the improvement looks like on the screen for a typical session — the session whose improvement is closest to the cohort median. Under eyelid-relative features many held-out predictions are displaced by long errors; under canthus-relative features the displacements shrink.

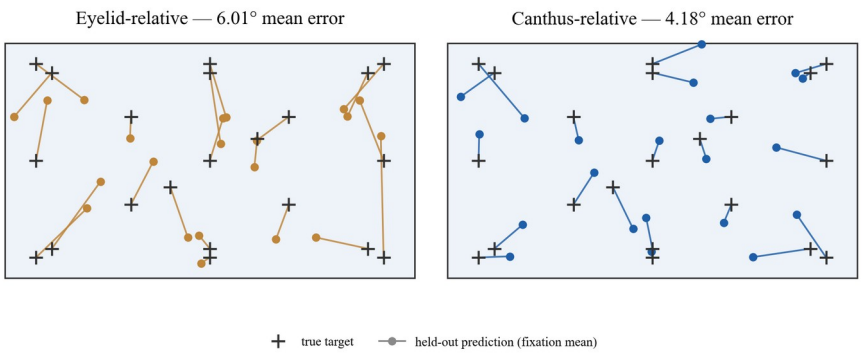


Figure 19 – Held-out predictions for one typical session (participant P02) under the two feature sets

Crosses are true targets; dots are the mean prediction for each target when that target was held out of the fit. Mean error falls from 6.01° to 4.18°.

Where the Gain Comes From: Error by Axis

If the eyelid-contamination mechanism of Section 2.2 is correct, the gain should appear in the *vertical* channel — the only one normalised by the eyelid aperture — and not in the horizontal one. This was tested directly for this report by splitting each held-out error into its horizontal and vertical components, computed with the same evaluation code and participant-level aggregation as the ablation (Figure 20).

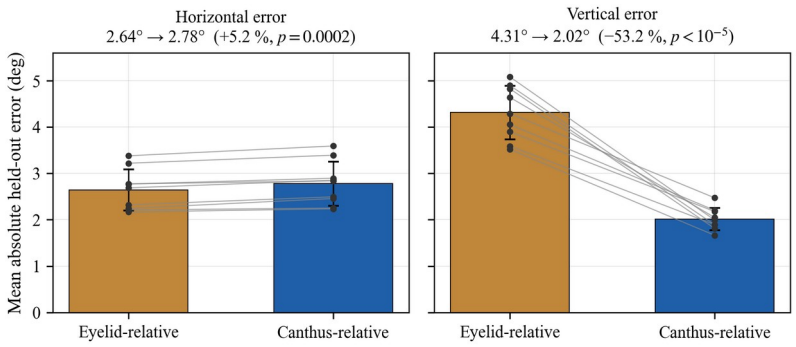


Figure 20 – Held-out error split by axis, participant level (each grey line is one participant; whiskers are one standard deviation)

The canthus gain is entirely vertical: the vertical error more than halves for every participant, while the horizontal error rises slightly.

The result is unambiguous. Under eyelid-relative features the vertical error (4.31°) is 1.6 times the horizontal error (2.64°) — the same asymmetry reported in the literature, here measured in degrees. Canthus normalisation reduces the vertical error to 2.02°, a 53.2 % reduction that every participant shares ($t(8) = 11.16$, $p = 3.7 \times 10^{-6}$). The horizontal error, by contrast, does not improve at all: it rises slightly, from 2.64° to 2.78° (+5.2 %, $t(8) = -6.35$, $p = 0.0002$), and does so for all nine participants. This is consistent with the canthus features adding little horizontal information — the conventional horizontal ratios are already normalised by the eye corners — while four extra features in a model fitted from few targets cost a small amount of variance on that axis.

This is the strongest evidence in the project for the eyelid mechanism: the improvement appears exactly where the mechanism predicts and nowhere else, although it remains supporting rather than causal evidence. It also identifies a concrete improvement for the next phase: fitting each screen axis with its own feature subset could recover the 5.2 % horizontal cost without giving up the vertical gain.

Roll Invariance and the Error Budget

Rotating a synthetic face through 0–40° and recomputing the features, the canthus components of Equation (3) drift by about 10^{-15} — floating-point precision — while an image-axis formulation of the same offset drifts by 0.069, a substantial fraction of the feature's own range. The representation remains sensitive to genuine iris movement, so the invariance is not achieved by the feature going dead.

This is presented as a property of the representation rather than as a measured accuracy gain, because the recordings cannot test the latter. Although the capture

protocol requested head tilts of ±15° on the hold-out targets, the achieved roll across all eleven sessions never exceeded 6.5° and its median was 1.9°: participants comply with an instruction to tilt far less than they are asked to — itself worth recording for anyone designing such a protocol. Within the range actually observed the representation neither helps nor hurts detectably.

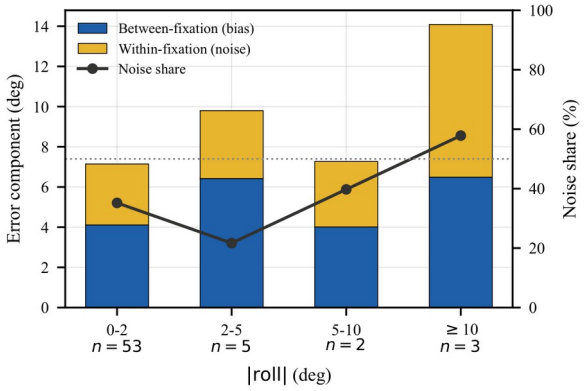


Figure 21 – Decomposition of gaze error by head roll on the pilot recordings (63 target groups)

Within-fixation noise is the scatter of predictions around their own mean, which the dwell centroid can average away; between-fixation bias is the distance from that mean to the target, which no averaging removes. The noise share grows with roll.

The pilot recordings, which did include larger head movements, show where the remaining error comes from (Figure 21). Each fixation's error splits into a *within-fixation* component — frame-to-frame scatter that the dwell centroid averages away — and a *between-fixation* component — a fixed offset that no averaging removes. At small roll the bias dominates; past 10° of roll the per-frame scatter grows about 5.5 times and becomes the larger term, although the high-roll bins hold few target groups.

Estimator Comparison and Model Selection

Table 18 places the deployed estimator against twenty alternatives spanning linear, robust, kernel, ensemble, neural and quantum-kernel families, all on the canthus features under the same leave-one-target-out protocol.

Table 18 – Estimators for the gaze coordinate, leave-one-target-out over 21 positions on the three-session pilot recording (the only set on which all arms were run)

Algorithm	Model family	Error (deg)	Change
Gaussian process (RBF + white)	Nonparametric	8.454	−92.8 %
Random forest	Ensemble	7.378	−68.2 %
Multilayer perceptron (32, 16)	Neural	6.397	−45.9 %
Ordinary least squares	Linear	4.282	+2.4 %
Bayesian ridge	Linear	4.274	+2.5 %
Huber regression	Robust linear	4.193	+4.4 %
Theil–Sen regression	Robust linear	4.107	+6.4 %
Support vector regression (RBF)	Kernel	4.107	+6.4 %
Polynomial (degree 2) + ridge	Expanded	4.087	+6.8 %
Kernel ridge (RBF)	Kernel	4.061	+7.4 %
Quantum kernel, ZZ map, 6 qubits	Quantum	10.281	−131.4 %
Quantum kernel, Z map, 8 qubits	Quantum	10.796	−143.0 %
Quantum kernel, ZZ map, 4 qubits, 2 repetitions	Quantum	15.735	−254.2 %
Proposed model (ridge, canthus-12)	**Linear**	**4.386**	**baseline**

Change is relative to the deployed ridge; quantum arms are compared with the ridge baseline of their own per-target subsample (4.443°).

Seven candidates beat the deployed ridge, the best of them — RBF kernel ridge — by 7.4 %. Reported at that, the table would recommend replacing the estimator. Two further tests say otherwise, and they are the reason the table is worth including.

The first is **leave-one-participant-out** with the model chosen inside each fold. Selecting on eight participants and scoring on the ninth, over all eleven sessions, the mean gain over the incumbent is +1.02 % and indistinguishable from zero:

$t(8)=0.19$, $p=0.85$, 95 % confidence interval $[-11.4\%, +13.4\%]$. The fold-by-fold gains are −4.1, +11.3, −1.5, +9.8, +4.2, +1.6, −38.2, +16.5 and +9.7 %. A margin whose confidence interval spans ±12 percentage points, and which costs one participant 38 %, cannot be deployed even where its point estimate is favourable. The second test is calibration size, which reverses the ranking outright (Section 5.4).

The **quantum-kernel arms** fail for an independent reason: fidelity kernels concentrate exponentially with qubit count [27]. Measured on these features, the mean off-diagonal kernel value falls from 0.128 at four qubits to 0.014 at twelve, so the Gram matrix approaches the identity and the model can only memorise, while a classical Gaussian kernel on the same features scores +0.8 %.

Model Class Against Calibration Size

The choice of regressor is usually reported at one training size, and on this data that is exactly the wrong way to report it: the ranking *inverts* between the size used for evaluation and the size the system actually deploys with. Four estimators — ridge, ordinary least squares, RBF kernel ridge with a 35-point hyperparameter grid, and a 16–8 multilayer perceptron — were compared on the canthus features. For each session and draw, *n* targets are sampled for training and every remaining target is held out; all four estimators receive the identical split, so the comparison is paired.

Table 19 – Mean gaze error (degrees) against the number of calibration targets used for fitting, canthus features, grouped by target

Targets	Ridge	OLS	Kernel ridge (RBF)	MLP	Best
4	**6.49**	10.52	9.52	15.92	Ridge
6	**5.47**	7.95	7.17	11.02	Ridge

Targets	Ridge	OLS	Kernel ridge (RBF)	MLP	Best
9 (deployed)	**4.47**	5.66	4.68	7.72	**Ridge**
12	4.37	4.60	**4.16**	5.89	Kernel ridge
16	3.90	4.23	**3.79**	5.21	Kernel ridge
20	3.87	3.99	**3.20**	4.54	Kernel ridge

Paired at nine targets over 165 draws: ridge 4.638 ± 1.097°; OLS +16.7 % ($p < 0.0001$); kernel ridge +2.9 % ($p = 0.90$, not significant); MLP +65.9 % ($p < 0.0001$).

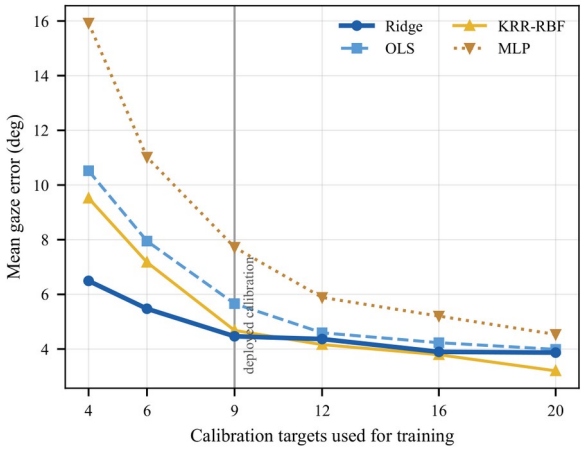


Figure 22 – Gaze error against the number of calibration targets

Ridge is the most accurate of the four at every size below twelve targets, including the nine the deployed system collects; kernel ridge overtakes it beyond that. Evaluating at a single training size would invert the recommendation.

At 20 training targets kernel ridge is the most accurate estimator and beats ridge by 17.2 % — the conclusion a conventional leave-one-target-out benchmark would reach. At the nine targets the deployed calibration actually collects, ridge is the most accurate of the four (Table 19, Figure 22). The mechanism is not subtle: kernel ridge must fit its regularisation and kernel width by cross-validation, and at nine targets the inner folds are fitting noise; at four targets it is 46 % worse than ridge and the perceptron 145 % worse. Ridge's regulariser is a fixed heuristic with nothing to estimate, so it degrades gracefully instead of collapsing.

Against kernel ridge the margin at nine targets is not significant, so the case for the linear model there rests on cost, and cost is decisive: a ridge refit costs 0.017 ms against 44.0 ms for kernel ridge, and per-sample inference 0.42 µs against 178 µs. A model-selection experiment run at a training size the deployed system never sees does not answer the question it appears to answer.

Calibration of the Commit Confidence

The threshold that decides whether to commit or to ask is applied to a number the system reports as a confidence, so that number should mean what it says: of the keystrokes committed at a stated confidence of 0.9, about nine in ten should be correct. Table 20 measures this directly on held-out keystrokes at σ = 40 px.

Table 20 – Calibration of the commit confidence on held-out keystrokes at σ = 40 px

Algorithm	ECE	Brier	AUC	Acc. @ 10 % prompts	Acc. @ 20 % prompts
Gradient boosting, all evidence	0.0528	0.0791	0.8606	0.9345	0.9568
Gaze posterior, uncalibrated	0.0650	0.0721	0.8908	0.9411	0.9705
Logistic, all commit evidence	0.0234	0.0672	0.8864	0.9383	0.9726
Temperature scaling (T = 0.55)	0.0229	0.0678	0.8908	0.9411	0.9705
Isotonic recalibration	0.0185	0.0663	0.8801	0.9411	0.9684
Logistic, top-p + margin	0.0225	0.0663	**0.8939**	0.9411	0.9726
Proposed (fused + Platt)	**0.0167**	0.0667	0.8908	**0.9411**	**0.9705**

Lower is better for ECE and Brier score, higher for AUC and accuracy.

The deployed fused posterior is over-confident, with an expected calibration error of 0.0650. Platt scaling reduces this to 0.0167, a factor of 3.9. Richer recalibrators that consume the full commit evidence do not improve on the simple monotone map, and gradient boosting is worse on almost every metric than the raw posterior it was meant to repair. The accuracy columns are the control: Platt scaling,

temperature scaling and the raw posterior commit *identically* at matched prompt rate, because a monotone transform cannot reorder candidates. Any movement there would indicate an implementation fault rather than a gain.

The Quantum Selection Rule at Matched Prompt Rate

Table 21 gives the held-out comparison between the quantum conditional selection rule and the deployed log-linear fusion. Tuned on the training split, the phase scale that maximised commit accuracy was $\sigma_\varphi = 0.40$, worth +0.03 percentage points there. On held-out sentences that advantage does not appear: across twenty (noise, prompt-rate) cells the median change is −0.03 percentage points, and a paired Wilcoxon test over sentences finds no significant difference at any of the five noise levels — zero wins and zero losses.

Table 21 – The selection rule against deployed log-linear fusion, held out, at matched prompt rate

σ (px)	2 % prompts	5 % prompts	10 % prompts	20 % prompts	p
30	0.971 / +0.003	0.977 / +0.002	0.986 / −0.001	0.988 / −0.000	1.00
40	0.921 / −0.002	0.932 / +0.001	0.958 / −0.000	0.973 / +0.003	0.80
50	0.888 / −0.002	0.897 / −0.001	0.914 / −0.004	0.945 / −0.001	0.34
60	0.841 / −0.000	0.853 / −0.000	0.862 / +0.001	0.891 / +0.008	0.80
75	0.778 / −0.000	0.795 / −0.003	0.819 / +0.001	0.851 / −0.004	0.80

Each cell gives the baseline commit accuracy and the rule's change. Median change over all 20 cells: −0.0003; significant wins 0, losses 0.

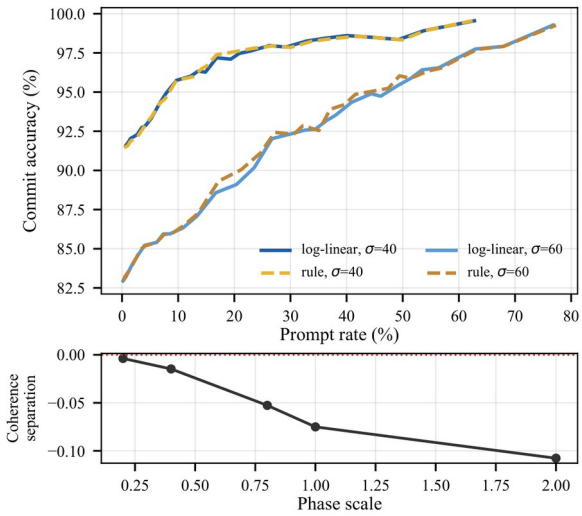


Figure 23 – Top: the selection rule traces the same accuracy/prompt-rate frontier as the log-linear fusion it generalises, at two noise levels

Bottom: the reason — the coherence separation between correct and incorrect letters is negative at every phase scale and falls monotonically.

The two frontiers lie on top of one another (Figure 23). The null is not an artefact of a rule that fails to engage — the posteriors differ, negligibly and without direction, by a mean Kullback–Leibler divergence of 4.2×10^{-4} nats per keystroke — nor of a scale chosen too small: scales above 0.6 produce substantial interference and *lose*, falling from 97.0 % to 95.3 % as σ_{φ} rises to 1.0.

Comparison With an Independent Classical Implementation

The comparisons above hold the implementation fixed and vary the rule. As a complementary check, an independent classical gaze-typing implementation was given the identical 60-sentence protocol, key geometry and noise levels. Figure 24 plots the resulting operating curves: commit accuracy against entry rate, with σ tracing each curve from 30 to 75 px.

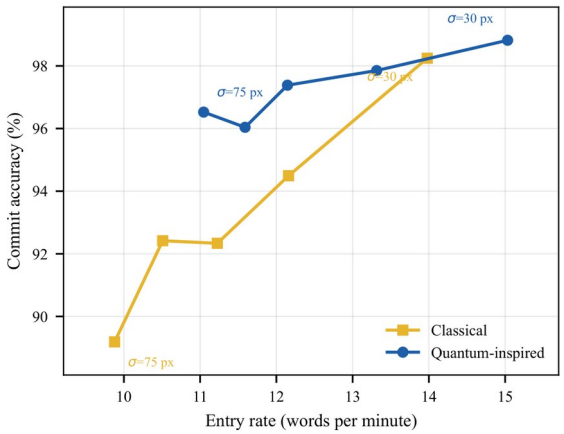


Figure 24 – Commit accuracy against entry rate for this system (blue, labelled "quantum-inspired") and an independent classical implementation (gold), on identical sentences, geometry and noise; points are the five noise levels

The gap is attributable to the front end and language model, not to the selection rule: this system's own log-linear control lies on the upper curve.

This system reaches 97.55 % pooled commit accuracy against 94.42 % and is ahead at every noise level. **The gap is not attributable to the quantum selection rule**: this system's own log-linear control — the same implementation with the phase channel disabled — scores 97.53 %, statistically indistinguishable from the quantum arm. What the comparison measures is the front end and the language model, and the control is reported precisely to rule out the natural misreading.

Amplitude Amplification and Dwell

Table 22 reports the number of frames needed to commit at σ = 40 px.

Table 22 – Frames to commit, held out, σ = 40 px

Rule	Median frames	Mean frames	Commit accuracy
Log-linear (deployed)	2.0	**2.63**	**0.984**
Interference only	2.0	2.66	0.988

Rule	Median frames	Mean frames	Commit accuracy
Amplification, before conditioning	2.0	2.80	0.964
Amplification, after conditioning	2.0	2.80	0.968
Interference + amplification	2.0	2.75	0.964

After conditioning, zero amplification rounds fire because the admissible subspace already holds 94–98 % of the amplitude; paired over the 249 keystrokes committed by both arms, p = 0.48.

Applied after the language-conditioned unitary, amplification is a **no-op by construction**: the conditioned state already places 94–98 % of its amplitude on consistent letters, so the optimal round count is zero (Section 5.4). Forced to act on the raw gaze register, where a scattered fixation has a marked mass of 0.31–0.78, it makes things worse: mean dwell rises from 2.63 to 2.80 frames and commit accuracy falls from 98.4 % to 96.4 %. The claim made at the first review, that amplitude amplification would shorten dwell time, has therefore been withdrawn.

Why Interference Fails: A Sign, Not a Magnitude

A null result is more useful with a mechanism attached, and this one has a sharp one. For interference to help, the coherence penalty must fall more heavily on wrong letters than on right ones. Table 23 measures exactly that separation.

Table 23 – Coherence |c| of the correct letter against the wrong ones

Phase scale σ_φ	Coherence of correct letter	Coherence of wrong letters	Separation
0.2	0.9934	0.9971	−0.0038
0.4 (selected on training split)	0.9740	0.9887	−0.0147
0.8	0.9046	0.9572	−0.0526
1.0	0.8607	0.9357	−0.0750
2.0	0.7691	0.8768	**−0.1076**

A positive separation would mean interference protects the correct letter; it is negative everywhere and worsens with scale.

The separation is negative at every phase scale and grows steadily more negative as the scale rises, from −0.004 at $\sigma_\varphi = 0.2$ to −0.108 at $\sigma_\varphi = 2.0$. The cause is structural. The correct letter following an English prefix is typically the one with the *richest* vocabulary support: many genuine continuations, therefore many hypotheses, therefore the widest spread of phases and the lowest coherence. A wrong letter is typically backed by one obscure completion and sits at coherence 1. Any phase computed from hypothesis likelihood thus penalises exactly the letter with the strongest linguistic evidence.

Two alternative explanations were tested and rejected: raising the hypothesis budget from 4 to 4,096 completions moves the mean coherence only from 0.982 to 0.988, and flattening the within-letter weights changes it from 0.9883 to 0.9879. The limitation is the phase *source*: a language model offers no feature orthogonal to the likelihood that already determines the magnitude. The result concerns likelihood-derived phase for English letter prediction, not the selection rule as a formal object.

Quantum Residual Gaze Correction

The residual correction stage was evaluated on the same protocol as the feature ablation — eleven sessions, nine participants, leave-one-target-out over 21 targets — with an adoption rule written into the benchmark before any result was seen: *the register enters the typing path only if it beats both the uncorrected ridge and a size-matched classical twin, each paired over participants at $p < 0.05$.* The classical twin is a conventional network with 109 parameters against the register's 102, trained on the same cross-fitted labels. Table 24 reports the five arms.

Table 24 – Quantum residual gaze correction against the uncorrected ridge and a size-matched classical twin, leave-one-target-out, nine participants

Arm	Error (deg)	Change vs none	Paired t-test p	Participants improved
Ridge only, no correction	3.765 ± 0.440	—	—	—
Quantum register, confidence-gated	3.747 ± 0.437	−0.5 %	0.0006	9 / 9
Quantum register, ungated	**3.607 ± 0.404**	**−4.2 %**	**0.0004**	**9 / 9**
Classical twin, gated	3.682 ± 0.444	−2.2 %	0.126	7 / 9
Classical twin, ungated	3.804 ± 0.467	+1.0 %	0.642	4 / 9

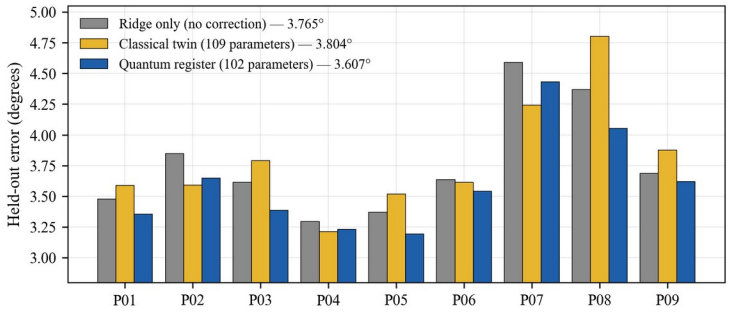


Figure 25 – Held-out error per participant for the uncorrected ridge, the classical twin and the quantum register (both ungated)

The register improves every participant.

The ungated register reduces error from 3.765° to 3.607°, a further 4.2 % on top of the canthus normalisation, and improves all nine participants (Wilcoxon $p = 0.0039$; Figure 25). Against the size-matched classical twin, however, the ungated register is 5.2 % better with $p = 0.071$ — better for six of nine participants but not significant at the pre-registered level. The adoption rule therefore fails, and the stage remains out of the typing path. Two further cautions apply: the ungated readout was chosen after evaluation, so it needs confirmation on new recordings, and the comparisons with classical kernel-ridge and support-vector residual models in the benchmark have not yet been run. The confidence-gated variant suppresses its own corrections — the mean scale factor is 0.06 — and gains only 0.5 %, so it is not preferred.

The result is reported as it stands: the register is a promising lead with a consistent, all-participant improvement, and the evidence needed to adopt it — a significant win over an equally sized classical model on fresh data — is the first task of Phase 2.

Results of the Hybrid Decision Layer

The remaining quantum components were evaluated partly by simulation and partly on recorded data, and Table 25 keeps the two clearly separated.

Table 25 – Results of the hybrid quantum–classical decision layer

Component	Experiment	Data	Result
QSVM movement gate	Eye-state classification, held-out split (n = 384)	Synthetic kinematics	Threshold rule 0.7817 → QSVM 0.9896; linear SVM 0.9896, MLP 0.9896, RBF-SVM 0.9870
QSVM gate + QBN reliability	Commit quality with 25 % of dwell windows corrupted, σ = 0.02	Synthetic	Wrong commits per 100: 12.67 (ungated) → 0.47 (gate) → 0.33 (gate + QBN); accuracy when committed 0.827 → 0.994; prompts 26.7 % → 44.4 %
QBN	Joint distribution against classical enumeration	Exact	Identical to 10^{-16}
QAOA word re-decode	Word accuracy, chip semantics	Simulated gaze noise	σ = 30 px: 80.0 % → 88.7 %; 40 px: 58.0 % → 61.3 %; 50 px: 40.0 % → 41.3 %
QAOA solver	Fixed-angle sampling against exact enumeration	Held-out segments	Matched the exact optimum on 27 / 27 segments
Quantum gaze→letter stack	Commit accuracy with the stack's factor enabled	660 real fixations	26.8 % → 26.1 % (−0.8 pp); quantum channels discarded on 100 % of decisions; removed

Synthetic and simulated results are valid for relative comparison only.

The movement gate's gain is over the hand-tuned threshold rule it replaced; a linear SVM achieves the identical accuracy, so the improvement comes from replacing a rule with any trained model, not from the quantum kernel. The gate and

reliability model trade prompts for correctness, a favourable trade for an assistive keyboard. Automatically accepting the QAOA word was *worse* than greedy decoding at σ = 40 px (−2.7 pp); offering it as a chip is what makes its contribution non-negative.

Live Typing Trials

Every result above was measured on recorded calibration sessions or in simulation. To test whether the results carry over to real typing, the live evaluation mode (`eval/live_eval.py`) was used for two short transcription trials by one participant before the third review. In this mode one rule decides each keystroke and the other is scored on the identical evidence at the same instant, so that the two arms see exactly the same gaze: in trial 1 the quantum selection rule drove the keystrokes, and in trial 2 the deployed classical fusion did. Table 26 reports both trials from the stored logs.

Table 26 – Live typing trials, one participant, logged by eval/live_eval.py (trial 1 from the committed log, trial 2 from the working log)

Measure	Trial 1	Trial 2
Rule driving the keystrokes	Quantum rule	Classical fusion
Typing time (s)	144.6	148.8
Characters committed	11	3
Correct commits (both arms)	2 (18.2 %)	1 (33.3 %)
Prompts offered	11 of 22 keystrokes (50.0 %)	2 of 5 keystrokes (40.0 %)
Entry rate	0.83 WPM	0.56 WPM
Mean model time per decision, classical fusion	0.017 ms	0.022 ms
Mean model time per decision, quantum rule	0.195 ms	0.226 ms

The two arms committed identical characters with identical accuracy in both trials, which is consistent with the matched-prompt-rate null; the quantum rule cost about ten times more computation per keystroke, which is negligible against a 693 ms dwell. The absolute numbers, however, are far below the 95–99 % commit

accuracy measured in simulation at σ = 30–40 px. Fourteen committed characters from one participant are far too few for a rate estimate, but the gap is too large to ignore, and the posture analysis that follows offers the most likely explanation.

Posture Drift After Calibration

A still-head calibration cannot learn what head position does to the gaze map. On the experimental dense-calibration branch (Section 4.2.2) one keyboard-protocol recording — all 28 keys fixated three times, then every key re-tested later in the session — was used to separate the error sources. With the head where it was calibrated, the 28-key calibration reached 2.07° with ridge regression (1.76° with a second-order polynomial and 2.49° with kernel ridge). After the participant's posture shifted, the error rose to about 10.6–10.8° for the polynomial and ridge models, and in the analysis of this recording 7.6° of an 8.3° error was a common offset shared by every estimate — an offset that no estimator can remove, because nothing in a still-head calibration describes it. Dropping the head-position and scale features halved the post-movement error of ridge (10.80° → 5.03°) at a cost in steady accuracy (2.07° → 3.13°).

Replayed on the same recording, the five-dot re-centre described in Section 4.2.2 reduced the error from 8.74° to 4.70° and raised the fraction of fixations landing on the intended key from 4 % to 26 %. These figures come from a single recording on an unmerged branch and are reported as a diagnosis rather than as a result: they identify posture, not the estimator, as the first-order error in live use, and they define the first task of the second phase.

Negative Results

Table 27 records every idea that was implemented, measured on recorded or simulated data, and rejected, so that later work need not repeat it.

Table 27 – Ideas that were implemented, measured and rejected

Idea	What was measured	Verdict
Iris-ring radius as a distance cue	+0.4 %, inside the noise	Dropped
Explicit head-pose features	+5.7 % alone, nothing on top of the canthus frame (p = 0.44)	Rejected — the canthus frame is already head-attached
Kernel ridge instead of linear	+7.4 % on a pooled split; +2.9 % (p = 0.90) at the deployed nine targets; +1.0 % (p = 0.85) under LOPO	Not deployed
Quantum-kernel regression	131–254 % worse than linear over 4 –10 qubits	Rejected — exponential kernel concentration
Phase interference between word hypotheses	Median −0.03 pp; 0 wins, 0 losses	Not deployed — the sign is backwards
Amplitude amplification	Dwell 2.63 → 2.80 frames; accuracy 98.4 % → 96.4 %	Not deployed — the subspace is not rare
Quantum gaze→letter stack	−0.8 pp on 660 real fixations; 68 ms per commit	Removed from the typing path
Two-level region selection	About 20 points more reliable per choice, but throughput fell from 18.5 to 14.4 correct letters per minute	Rejected
De-rotating the eye region before detection	No perceptible gain in live use	Reverted — the detector appears to align internally
Confidence-gated residual correction	Mean scale 0.06, gain 0.5 %	Not preferred

Where Quantum Helped and Where It Did Not

Table 28 brings every quantum and quantum-inspired component together with its classical counterpart, its measured outcome and its status in the deployed system.

Table 28 – Scorecard of the quantum and quantum-inspired components

Component	Quantum mechanism	Classical counterpart	Outcome	In typing path
Selection rule	Interference over a 5-qubit register	Log-linear fusion (contained exactly)	Null: −0.03 pp; mechanism diagnosed	Research module

Component	Quantum mechanism	Classical counterpart	Outcome	In typing path
Amplitude amplification	Grover-type rotation	None	Lengthens dwell, costs accuracy	No
Residual correction	6-qubit register, expected-value readout	Classical twin (109 parameters)	−4.2 % vs ridge, 9/9; vs twin +5.2 %, p = 0.071	Pending confirmation
Kernel regression	ZZ / angle fidelity kernels	Ridge, kernel ridge	131–254 % worse	No
Movement gate	Fidelity-kernel SVM	Threshold rule; linear SVM	+20.8 pp over the rule; ties linear SVM	Yes
Reliability	Exact quantum Bayesian network	Classical Bayesian network	Identical by construction; useful as a model	Yes
Word re-decoding	QAOA with XY mixer, exact fallback	Exact chain MAP	Matches exact; +8.7 pp at σ = 30 px as chips	Yes, as chips
Gaze→letter stack	VQC, QSVM, quantum kernel, QAOA consensus	Exact Gaussian posterior	−0.8 pp on real data	Removed

The pattern in the scorecard is the most important lesson of the project. Where a quantum component competed with a classical stage that is already *exact* — the closed-form ridge, the exact Gaussian key posterior, exact Bayesian-network enumeration — it could at best tie, and it did. Where it was compared with a weak incumbent such as a hand-tuned threshold, it won, but so did an equally simple classical learner. Where the quantum formulation was genuinely richer — the selection rule — the extra expressiveness pointed the wrong way. The components that earned a place did so through structure rather than through quantumness, and the accuracy of the system came from the front end: the four canthus features are worth more than every decision-side component combined, at no additional runtime cost.

Worked Calculations

The calculations behind the principal numbers above are worked here from the recorded session geometry.

Pixels per degree of visual angle. A visual angle of 1° at viewing distance D subtends $s = 2D\tan(0.5^\circ)$ on the screen. With $D = 40$ cm,

$$s = 2 \times 40 \times \tan(0.5^\circ) = 80 \times 0.0087269 = 0.6981\,\text{cm}. \tag{13}$$

The display has $1470\,\text{px}/29.4\,\text{cm} = 50$ px/cm, so one degree is $0.6981 \times 50 = 34.91$ px, matching the value stored in every session record.

Error in degrees and in key pitches. The deployed eyelid-relative features have a held-out error of 193.8 px, and the canthus features 131.3 px. Converting,

$$\frac{193.8}{34.91} = 5.55^\circ, \frac{131.3}{34.91} = 3.76^\circ. \tag{14}$$

The key pitch on this display is 145.6 px, or $145.6/34.91 = 4.17^\circ$. The conventional features therefore err by $193.8/145.6 = 1.33$ key pitches — more often onto a neighbouring key than the intended one — while the canthus features err by $131.3/145.6 = 0.90$ key pitches, for the first time below a single key.

Reading a canthus feature. In Table 12 the left eye has $c_{\parallel}^{L} = 0.0055$ and $c_{\perp}^{L} = -0.1407$: the iris sits almost midway between its canthi along the eye axis and 14 % of the inter-canthal distance away from the canthal line. Both are dimensionless fractions of the eye's own width and involve no eyelid landmark.

The paired test. Across the nine participants the mean reduction in error is $\acute{d} = 1.791^\circ$ with $t(8) = 9.31$. The standard deviation of the paired differences is therefore

$$s_d = \frac{\acute{d}\sqrt{n}}{t} = \frac{1.791 \times 3}{9.31} = 0.577^\circ, \tag{15}$$

the effect size is $d_z = \acute{d}/s_d = 3.10$, and the 95 % confidence interval is $\acute{d} \pm t_{0.975,8}\, s_d/\sqrt{n} = 1.791 \pm 2.306 \times 0.577/3 = [\,1.35^\circ, 2.23^\circ\,]$, matching the values reported in Section 5.4.

Correction hypotheses. With eight hypotheses 1.5° apart, Equation (12) gives centres $c_i = (i - 3.5) \times 1.5^\circ$, that is −5.25°, −3.75°, −2.25°, −0.75°, +0.75°, +2.25°, +3.75° and +5.25°. A residual of +1.2° lies between the hypotheses at +0.75° and +2.25°; its soft label puts 0.7 on the first and 0.3 on the second, and the expectation $0.7 \times 0.75 + 0.3 \times 2.25 = 1.20^\circ$ reproduces the residual exactly.

Why amplitude amplification does not fire. After language conditioning the admissible subspace holds 94–98 % of the amplitude. With $p_S = 0.94$, $\arcsin\sqrt{0.94} = 1.323$ rad, so $m = \lfloor 0.785/1.323 \rfloor = \lfloor 0.59 \rfloor = 0$ rounds — the operator is never applied. On the raw gaze register a scattered fixation has p_S as low as 0.31, giving $\arcsin\sqrt{0.31} = 0.590$ rad and $m = \lfloor 1.33 \rfloor = 1$ round; that is the case Section 5.4 measures.

REFERENCE :

[1] P. Majaranta and K.-J. Räihä, "Twenty years of eye typing: Systems and design issues," in *Proc. Symp. Eye Tracking Research and Applications (ETRA)*, New Orleans, LA, USA, 2002, pp. 15–22.

[2] D. W. Hansen and Q. Ji, "In the eye of the beholder: A survey of models for eyes and gaze," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 32, no. 3, pp. 478–500, Mar. 2010.

[3] R. J. K. Jacob, "The use of eye movements in human-computer interaction techniques: What you look at is what you get," *ACM Trans. Inf. Syst.*, vol. 9, no. 2, pp. 152–169, Apr. 1991.

[4] X. Zhang, Y. Sugano, M. Fritz, and A. Bulling, "Appearance-based gaze estimation in the wild," in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, Boston, MA, USA, 2015, pp. 4511–4520.

[5] K. Krafka, A. Khosla, P. Kellnhofer, H. Kannan, S. Bhandarkar, W. Matusik, and A. Torralba, "Eye tracking for everyone," in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, Las Vegas, NV, USA, 2016, pp. 2176–2184.

[6] K. A. Funes Mora, F. Monay, and J.-M. Odobez, "EYEDIAP: A database for the development and evaluation of gaze estimation algorithms from RGB and RGB-D cameras," in *Proc. Symp. Eye Tracking Research and Applications (ETRA)*, Safety Harbor, FL, USA, 2014, pp. 255–258.

[7] V. I. Saraswati, R. Sigit, and T. Harsono, "Eye gaze system to operate virtual keyboard," in *Proc. Int. Electronics Symp. (IES)*, Denpasar, Indonesia, Sep. 2016, pp. 175–179, doi: 10.1109/ELECSYM.2016.7860997.

[8] D. A. Padilla, J. A. B. Adriano, J. R. Balbin, I. G. Matala, J. J. R. Nicolas, and S. R. R. Villadelgado, "Implementation of eye gaze tracking technique on FPGA-based on-screen keyboard system using Verilog and MATLAB," in *Proc. IEEE Region 10 Conf. (TENCON)*, Penang, Malaysia, Nov. 2017, pp. 2771–2776, doi: 10.1109/TENCON.2017.8228333.

[9] M. R. Islam, M. S. Rahman, and A. Sarkar, "Computer vision based eye gaze controlled virtual keyboard for people with quadriplegia," in *Proc. Int. Conf. Automation, Control and Mechatronics for Industry 4.0 (ACMI)*, Rajshahi, Bangladesh, Jul. 2021, pp. 1–6, doi: 10.1109/ACMI53878.2021.9528213.

[10] V. Abhaya, A. S. Bharadwaj, C. C. Bagan, K. Dhanraj, and G. Shyamala, "Eye-Move: An eye gaze typing application with OpenCV and Dlib library," in *Proc. Int. Conf. Automation, Computing and Renewable Systems (ICACRS)*, Dec. 2022, pp. 952–957, doi: 10.1109/ICACRS55517.2022.10029276.

[11] B. G. Varshini, A. Manimegalai, S. D. Abirami, and S. Kalaiselvi, "Eye-gaze controlled virtual keyboard using blink detection for accessible hands-free communication," in *Proc. Int. Conf. Computational, Communication and Information Technology (ICCCIT)*, Feb. 2025, pp. 49 –56, doi: 10.1109/ICCCIT62592.2025.10927769.

[12] K. Kathpal, S. Negi, and S. Sharma, "iChat: Interactive eyes for specially challenged people using OpenCV Python," in *Proc. 9th Int. Conf. Reliability, Infocom Technologies and Optimization (ICRITO)*, Noida, India, Sep. 2021, pp. 1–5, doi: 10.1109/ICRITO51393.2021.9596502.

[13] T. Soukupová and J. Čech, "Real-time eye blink detection using facial landmarks," in *Proc. 21st Computer Vision Winter Workshop (CVWW)*, Rimske Toplice, Slovenia, 2016.

[14] S. T. S. Rafid, A. J. Ajwad, and A. K. M. A. M. Azad, "Empowering quadriplegic patients: A computer vision based approach for keyboard control using eye gaze and blink," in *Proc. 26th Int. Conf. Computer and Information Technology (ICCIT)*, Cox's Bazar, Bangladesh, Dec. 2023, pp. 1–5, doi: 10.1109/ICCIT60459.2023.10441390.

[15] A. SK, E. D. A, H. B, and H. M, "Eye gaze tracking for virtual mouse control using computer vision," in *Proc. 4th Int. Conf. Artificial Intelligence and Machine Learning Applications (AIMLA)*, Apr. 2026, pp. 1–6, doi: 10.1109/AIMLA67915.2026.11522321.

[16] C. Lugaresi *et al.*, "MediaPipe: A framework for building perception pipelines," 2019, *arXiv:1906.08172*.

[17] B. Waideman and P. T. Aquino Junior, "Augmentative and alternative communication using eye tracking and word recommendation using language models," *IEEE Latin America Transactions*, vol. 23, no. 8, pp. 637–645, Aug. 2025, doi: 10.1109/TLA.2025.11072932.

[18] L. Sesma, A. Villanueva, and R. Cabeza, "Evaluation of pupil center-eye corner vector for gaze estimation using a web cam," in *Proc. Symp. Eye Tracking Research and Applications (ETRA)*, Santa Barbara, CA, USA, 2012, pp. 217–220.

[19] A. Papoutsaki, P. Sangkloy, J. Laskey, N. Daskalova, J. Huang, and J. Hays, "WebGazer: Scalable webcam eye tracking using user interactions," in *Proc. 25th Int. Joint Conf. Artificial Intelligence (IJCAI)*, New York, NY, USA, 2016, pp. 3839–3845.

[20] D. Li, L. Zhang, W. Hu, H. Li, and X. Zhou, "Dwell time adaptive three-way text entry by gaze," in *Proc. 31st Chinese Control and Decision Conf. (CCDC)*, Nanjing, China, Jun. 2019, pp. 5295–5300, doi: 10.1109/CCDC.2019.8832924.

[21] D. D. Salvucci and J. H. Goldberg, "Identifying fixations and saccades in eye-tracking protocols," in *Proc. Symp. Eye Tracking Research and Applications (ETRA)*, Palm Beach Gardens, FL, USA, 2000, pp. 71–78.

[22] P.-O. Kristensson and S. Zhai, "SHARK²: A large vocabulary shorthand writing system for pen-based computers," in *Proc. ACM Symp. User Interface Software and Technology (UIST)*, Santa Fe, NM, USA, 2004, pp. 43–52.

[23] R. W. Soukoreff and I. S. MacKenzie, "Metrics for text entry research: An evaluation of MSD and KSPC, and a new unified error metric," in *Proc. ACM CHI Conf. Human Factors in Computing Systems*, Ft. Lauderdale, FL, USA, 2003, pp. 113–120.

[24] L. K. Grover, "A fast quantum mechanical algorithm for database search," in *Proc. 28th Annu. ACM Symp. Theory of Computing (STOC)*, Philadelphia, PA, USA, 1996, pp. 212–219.

[25] G. Brassard, P. Høyer, M. Mosca, and A. Tapp, "Quantum amplitude amplification and estimation," in *Quantum Computation and Information*, ser. AMS Contemporary Mathematics, vol. 305, 2002, pp. 53–74.

[26] V. Havlíček, A. D. Córcoles, K. Temme, A. W. Harrow, A. Kandala, J. M. Chow, and J. M. Gambetta, "Supervised learning with quantum-enhanced feature spaces," *Nature*, vol. 567, no. 7747, pp. 209–212, Mar. 2019.

[27] S. Thanasilp, S. Wang, M. Cerezo, and Z. Holmes, "Exponential concentration in quantum kernel methods," *Nature Communications*, vol. 15, art. no. 5200, Jun. 2024.

[28] A. Pérez-Salinas, A. Cervera-Lierta, E. Gil-Fuster, and J. I. Latorre, "Data re-uploading for a universal quantum classifier," *Quantum*, vol. 4, p. 226, Feb. 2020.

[29] J. R. McClean, S. Boixo, V. N. Smelyanskiy, R. Babbush, and H. Neven, "Barren plateaus in quantum neural network training landscapes," *Nature Communications*, vol. 9, art. no. 4812, Nov. 2018.

[30] V. Bergholm *et al.*, "PennyLane: Automatic differentiation of hybrid quantum-classical computations," 2018, *arXiv:1811.04968*.

[31] E. Farhi, J. Goldstone, and S. Gutmann, "A quantum approximate optimization algorithm," 2014, *arXiv:1411.4028*.

[32] S. Hadfield, Z. Wang, B. O'Gorman, E. G. Rieffel, D. Venturelli, and R. Biswas, "From the quantum approximate optimization algorithm to a quantum alternating operator ansatz," *Algorithms*, vol. 12, no. 2, p. 34, Feb. 2019.

[33] J. C. Platt, "Probabilistic outputs for support vector machines and comparisons to regularized likelihood methods," in *Advances in Large Margin Classifiers*, A. J. Smola, P. Bartlett, B. Schölkopf, and D. Schuurmans, Eds. Cambridge, MA, USA: MIT Press, 1999, pp. 61 –74.

[34] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger, "On calibration of modern neural networks," in *Proc. 34th Int. Conf. Machine Learning (ICML)*, Sydney, Australia, 2017, pp. 1321–1330.

[35] A. E. Hoerl and R. W. Kennard, "Ridge regression: Biased estimation for nonorthogonal problems," *Technometrics*, vol. 12, no. 1, pp. 55–67, Feb. 1970.

[36] G. Casiez, N. Roussel, and D. Vogel, "1 € filter: A simple speed-based low-pass filter for noisy input in interactive systems," in *Proc. ACM CHI Conf. Human Factors in Computing Systems*, Austin, TX, USA, 2012, pp. 2527–2530.

[37] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *Proc. 3rd Int. Conf. Learning Representations (ICLR)*, San Diego, CA, USA, 2015.